



Corso di Laurea Magistrale in Informatica

Appunti di Big Data Processing and Analytics

Anno Accademico 2024/2025

Studente: Emmanuel Messina

Politecnico Di Torino

Indice

1	Introduzione ai Big Data	5
1.1	Dati in generale	5
1.2	Architettura Single Node (Nodo Singolo)	5
1.2.1	Caratteristiche principali:	5
1.2.2	Vantaggi:	6
1.2.3	Svantaggi:	6
1.3	Architettura Cluster	6
1.3.1	Caratteristiche principali:	6
1.3.2	Tipologie di nodi in un cluster:	7
1.3.3	Vantaggi:	7
1.3.4	Svantaggi:	7
1.4	Quando usare una o l'altra?	8
1.5	Architettura Lambda	8
1.5.1	Componenti principali:	8
1.5.2	Vantaggi:	9
1.5.3	Svantaggi:	9
2	Hadoop e MapReduce	10
2.1	Introduzione a Hadoop	10
2.1.1	Hadoop Distributed File System (HDFS)	10
2.1.2	MapReduce	11
2.1.3	YARN (Yet Another Resource Negotiator)	11
2.1.4	Hadoop Common	11
2.1.5	Vantaggi di Hadoop	11
2.1.6	Limitazioni di Hadoop	12
2.1.7	Conclusione	12
2.1.8	Esempio pratico di elaborazione distribuita con Hadoop e MapReduce	12
2.1.9	Scalabilità	13
2.1.10	Considerazioni su Hadoop/MapReduce	14
2.2	MapReduce su Java	14
2.2.1	Funzioni Map e Reduce	14
2.2.2	Esecuzione Coordinata da Hadoop	14
2.2.3	Componenti del Programma Hadoop MapReduce	15
2.2.4	Esecuzione di Job e Task	15
2.2.5	Fasi di MapReduce	15
2.2.6	Distribuzione e Parallelizzazione	15
2.3	Programmare MapReduce in Java	16
2.3.1	Tipi di Dati in Hadoop	16
2.3.2	InputFormat	17
2.3.3	OutputFormat	18
2.3.4	Driver	18
2.3.5	Mapper	19
2.3.6	Reducer	20
2.4	Combiner in Hadoop	20

2.4.1	Funzione <code>reduce()</code>	20
2.4.2	Esempio di Combiner in Hadoop	21
2.5	Tipi di Dati Personalizzati	21
2.5.1	Implementazione di <code>Writable</code>	21
2.5.2	Esempio: Classe <code>SumAndCountWritable</code>	21
2.6	Configurazione di Hadoop	22
2.6.1	Personalizzazione della Configurazione	22
2.7	Contatori Personalizzati	22
2.7.1	Esempio di Contatore Personalizzato	23
2.8	Map-Only Jobs	23
2.9	In-Mapper Combiner	23
3	Lettura di Dati da Dataset Multipli in Hadoop	24
3.1	Esempio di Applicazione con Dati da Sensori Diversi	24
3.2	Configurazione degli Input Multipli nel Driver	24
3.3	Distributed Cache in Hadoop	24
3.4	Utilizzo del Distributed Cache	25
3.5	Efficienza e Utilizzo della Cache Distribuita	25
3.6	Esempio di Codice per la Gestione della Distributed Cache	25
4	Introduzione ai Pattern di MapReduce	27
4.1	Sommatorie Numeriche	27
4.2	Indice Invertito	27
4.3	Filtraggio	28
4.4	Top K	28
4.5	Selezione di Valori Distinti	29
4.6	Shuffle (Riordino Casuale)	29
4.7	Job Chaining	29
4.8	Join	30
4.8.1	Esempio di join lato reduce:	30
4.8.2	Esempio di join lato map:	30
5	Introduzione ad Apache Spark	32
5.1	Esecuzione Iterativa: Confronto tra MapReduce e Spark	32
5.2	Problema con MapReduce	32
5.3	Soluzione proposta da Spark	32
5.4	RDD (Resilient Distributed Dataset)	32
5.5	Caratteristiche principali degli RDD	32
5.6	Creazione di un RDD	33
5.7	Programmazione in Spark: Operazioni sugli RDD	33
5.8	Componenti principali di Spark	33
5.9	Esempio di Word Count con Spark	33
5.10	Ecosistema Spark: Schedulazione e Parallelismo	34

6	RDD-Resilient Distributed Dataset	35
6.1	Spark Context	35
6.2	RDD - Resilient Distributed Dataset	35
6.3	Creazione di un RDD	35
6.4	Tipi di Operazioni su un RDD	36
7	Spark in Python	37
7.1	Funzioni Lambda	37
7.2	Trasformazioni su Spark	38
7.2.1	filter	38
7.2.2	map	38
7.2.3	flatMap	39
7.2.4	distinct	39
7.2.5	sortBy	39
7.2.6	Introduzione alle trasformazioni tra due RDD	39
7.3	Azioni su Spark	41
7.4	RDD basati su coppie chiave-valore	44
7.4.1	Trasformazioni di RDD di coppie chiave-valore	45
7.4.2	Trasformazioni tra due RDD	47
7.4.3	Azioni di RDD di coppie chiave-valore	48
8	Persistenza e cache	50
8.1	Utilizzo di persist e cache	51
8.2	Rimozione manuale e automatica dei dati	51
8.3	Accumulatori	52
9	Spark SQL	53
9.1	DataFrames	53
9.2	Creare un DataFrames	53
9.3	Operazioni sui Dataframes	55
9.4	Funzioni di aggregazioni	56
9.5	Dataframe e SQL	57
9.6	Approcci alla Memorizzazione	58
9.7	Funzioni Predefinite e Funzioni Definite dall'Utente (UDF)	58
9.8	Operazioni di Set sui DataFrame	59
9.9	Join e Broadcast	59
9.10	Piano di Esecuzione	59
9.11	Conclusioni e Recensione Finale	60
10	Apache Spark MLlib	61
10.1	Rappresentazione dei dati	61
10.2	Trasformazione dei dati grezzi	61
10.3	Pipeline di machine learning	62
10.4	Feature Transformations	63
10.4.1	Vector Assembler	63
10.5	Normalizzazione	64
10.5.1	Standard Scaler	64

10.6 Gestione delle variabili categoriali	65
10.7 SQLTransformer	66
10.8 Algoritmi di classificazione	67
10.8.1 Input per i modelli di classificazione	67
10.8.2 Esempio: classificazione dei clienti	68
10.8.3 Logistic Regression	69
10.8.4 Predizione su nuovi dati non etichettati	70

Premessa Prima di iniziare a studiare da questi appunti è necessario che tu sappia che, nonostante si sia cercato di fare gli appunti nel miglior modo possibile sia per ripassare al meglio e sia per poter creare un materiale utile per altri studenti, che ci potrebbero ugualmente essere imprecisioni, errori di scrittura o altro. In caso di problemi si prega di contattarmi via mail: s333951@studenti.polito.it, oppure il mio insta @emmanuelmessina00 oppure ancora sul canale telegram: **CLICCA QUI** dove potrai trovare gli altri appunti del primo semestre del primo anno di Ingegneria Informatica Magistrale.

1 Introduzione ai Big Data

1.1 Dati in generale

Abbiamo diversi tipi di dati. Su internet abbiamo dati di video, immagini, testo e altro. Non è l'unica tipologia di dati. Ci sono dati generati da utenti, possono essere dati generati da ricerche, dati dei server o di dispositivi connessi ad internet (IoT). Con **big data** si intendono i dati che contengono una maggiore varietà e che arrivano in volumi crescenti e con più velocità. Le 5 V dei Big data sono un concetto che rappresenta le caratteristiche principali dei dati generati in grandi quantità e velocità. Le 5 V dei Big data sono volume, varianza, velocità, veridicità e valore.

- Il **volume** si riferisce alla vasta quantità di dati generati. I Big data si contraddistinguono per le loro dimensioni, che possono essere molto superiori rispetto ai dati tradizionali. La gestione e l'analisi di grandi volumi di dati richiedono infrastrutture e strumenti appositi.
- La **velocità** indica la velocità con cui avviene la generazione e raccolta dei dati. Nel contesto dei Big data, i dati possono essere generati in tempo reale o in alta frequenza. La capacità di elaborare e analizzare i dati in real-time diventa quindi cruciale. Lo scopo è di ottenere informazioni utili e reattività nelle decisioni aziendali.
- La **varianza** riguarda la diversità dei tipi di dati presenti nel contesto dei Big data. I dati possono provenire da diverse fonti, come social media, sensori, dispositivi mobili, database aziendali, e possono essere: strutturati, non strutturati o semi-strutturati. La gestione e l'analisi di questa varietà di dati richiede soluzioni flessibili e adattabili.
- La **veridicità** concerne la qualità e l'affidabilità dei dati. Nel contesto dei Big data, è importante garantire che i dati raccolti siano accurati, completi e privi di errori. È necessario implementare processi di controllo della qualità per assicurarsi che i dati siano affidabili e che le informazioni ottenute siano valide.
- Il **valore** infine rappresenta il potenziale valore che può essere tratto dai dati. I Big data offrono l'opportunità di analizzare e sfruttare i dati per ottenere informazioni preziose, identificare tendenze, modelli e correlazioni, migliorare la presa decisionale, individuare nuove opportunità di business e offrire un'esperienza personalizzata ai clienti.

La quantità di dati aumenta ogni giorno. Per analizzarli c'è bisogno di sistemi speciali. Degli errori sono parte della vita di tutti i giorni, specialmente nei data center dovuti a errori di hardware/software, problemi elettrici. Come soluzione si può puntare a programmi/codici piccoli e all'utilizzo di dischi distribuiti per dividere i dati.

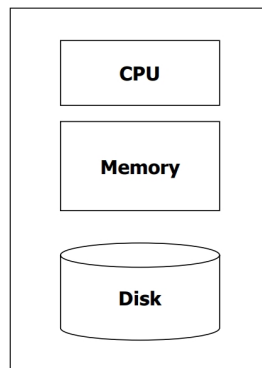
1.2 Architettura Single Node (Nodo Singolo)

Nell'architettura **single node**, tutto il carico di lavoro, sia per l'elaborazione che per l'archiviazione, viene gestito da un singolo server o macchina. Questo tipo di configurazione viene solitamente utilizzato per dataset di dimensioni più piccole o per ambienti di sviluppo e test.

1.2.1 Caratteristiche principali:

- **Risorse centralizzate:** CPU, RAM, storage e rete sono concentrati su una sola macchina.

Server (Single node)



- **Facilità di configurazione:** Essendo una singola macchina, la configurazione è più semplice rispetto a un cluster.
- **Limitazioni di scalabilità:** Poiché tutto viene eseguito su una singola macchina, la scalabilità è limitata alle risorse hardware disponibili.
- **Singolo punto di guasto (SPOF):** Se la macchina fallisce, l'intero sistema smette di funzionare, poiché non ci sono altri nodi per il failover.

1.2.2 Vantaggi:

- **Semplicità:** Non richiede la gestione di più nodi, quindi è più semplice da mantenere.
- **Minor costo:** È inizialmente più economico poiché si necessita di un solo server.
- **Adatto per test o piccoli dataset:** Ideale per piccoli volumi di dati o ambienti di sviluppo/test.

1.2.3 Svantaggi:

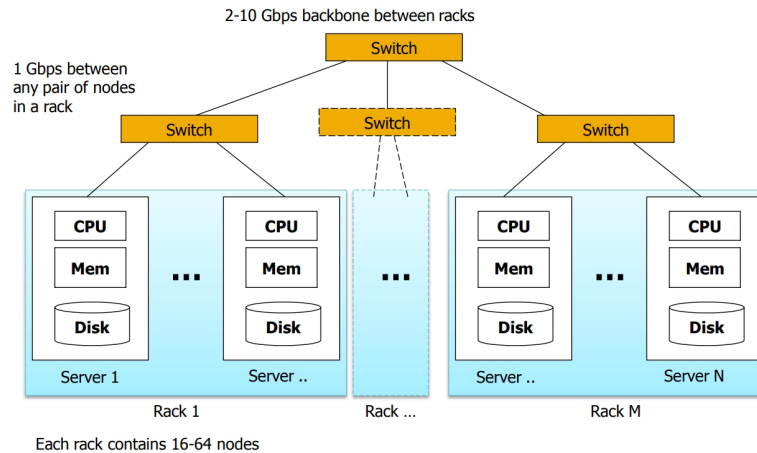
- **Scalabilità limitata:** Non adatto per grandi quantità di dati.
- **Affidabilità ridotta:** Se il server va in crash, non c'è ridondanza.

1.3 Architettura Cluster

Nell'architettura **cluster**, più macchine o nodi lavorano insieme per eseguire attività di elaborazione e archiviazione. Questo modello è comune negli ambienti di **Big Data**.

1.3.1 Caratteristiche principali:

- **Distribuzione del carico di lavoro:** I dati e le attività vengono suddivisi tra diversi nodi, consentendo un'elaborazione parallela.



- **Scalabilità orizzontale:** È possibile aggiungere nuovi nodi al cluster per aumentare la capacità di elaborazione.
- **Affidabilità:** Il fallimento di un nodo non compromette l'intero sistema, poiché i dati possono essere ridistribuiti su altri nodi.
- **Infrastruttura più complessa:** Richiede tecnologie avanzate per la gestione delle risorse e dei guasti.

1.3.2 Tipologie di nodi in un cluster:

- **Nodi Master:** Coordinano le operazioni del cluster, gestendo il traffico di dati e assegnando compiti ai nodi lavoratori.
- **Nodi Worker:** Eseguono l'elaborazione dei dati, gestendo i compiti specifici e immagazzinando i dati.

1.3.3 Vantaggi:

- **Scalabilità:** Aggiungere nuovi nodi permette di espandere la capacità senza sostituire l'hardware esistente.
- **Affidabilità e tolleranza ai guasti:** Se un nodo fallisce, il lavoro può essere ridistribuito.
- **Elaborazione parallela:** I lavori possono essere eseguiti su più nodi in parallelo, migliorando le prestazioni.

1.3.4 Svantaggi:

- **Maggiore complessità:** Richiede una configurazione e una gestione più complesse rispetto a un singolo nodo.
- **Costi superiori:** Mantenere un cluster di macchine è più costoso rispetto a una singola macchina.

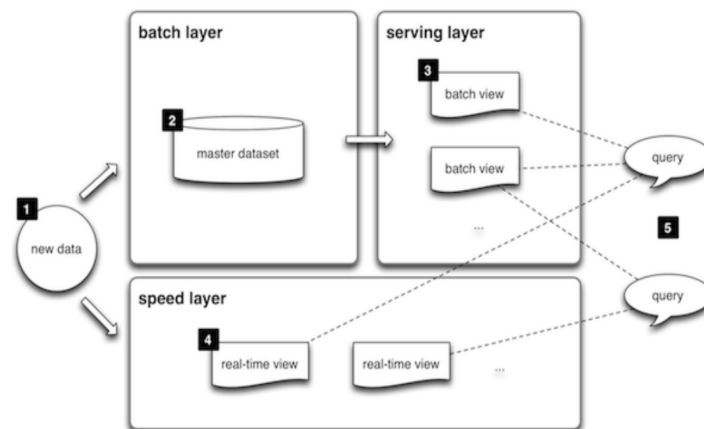
1.4 Quando usare una o l'altra?

- **Single Node:** Adatto per ambienti di sviluppo, test, applicazioni con dataset piccoli.
- **Cluster:** Ideale per ambienti di produzione su larga scala con grandi quantità di dati.

In sintesi, l'architettura **single node** è semplice e adatta a piccoli carichi di lavoro, ma non è scalabile o affidabile come un'architettura **cluster**, che è progettata per gestire grandi volumi di dati e offrire maggiore tolleranza ai guasti.

1.5 Architettura Lambda

Le soluzioni di big data possono affrontare diverse esigenze di elaborazione, dalla raccolta di dati storici all'analisi in tempo reale, fino alla scoperta di informazioni e previsioni future. Queste differenti modalità di lavoro consentono alle organizzazioni di trarre valore dai dati in vari contesti e applicazioni. L'architettura più frequentemente utilizzata per i big data è la Lambda Architecture. Questa architettura è stata proposta da Nathan Marz nel 2011. La Lambda Architecture è progettata per gestire grandi volumi di dati in modo efficiente, combinando un percorso batch per l'elaborazione di grandi quantità di dati storici con un percorso in tempo reale per i dati che arrivano continuamente. Questo approccio consente di ottenere analisi rapide e accurate su dati sia storici che in tempo reale.



1.5.1 Componenti principali:

- **Batch Layer:** Si occupa di elaborare e archiviare i dati storici, creando viste globali che vengono utilizzate per l'analisi. I dati in questa fase vengono elaborati in modalità batch, ossia in blocchi di dati che vengono analizzati tutti insieme a intervalli regolari.
- **Speed Layer:** Elabora i dati in tempo reale o near real-time per fornire informazioni immediate, utilizzando tecnologie di streaming che permettono l'analisi di dati man mano che vengono generati.

- **Serving Layer:** Combina i risultati elaborati dalle due precedenti componenti, creando viste che uniscono i dati storici con quelli più recenti per offrire un quadro completo e aggiornato delle informazioni.

1.5.2 Vantaggi:

- **Combina analisi batch e real-time:** Offre una soluzione bilanciata che gestisce sia grandi volumi di dati storici sia flussi continui di dati.
- **Alta disponibilità:** Grazie alla separazione delle componenti, l'architettura è resiliente ai guasti e può continuare a funzionare anche se una parte del sistema si arresta.

1.5.3 Svantaggi:

- **Complessità:** Implementare e mantenere una Lambda Architecture può essere complesso, poiché richiede la gestione di due pipeline di elaborazione dati distinte.
- **Doppio lavoro:** Spesso lo stesso dato viene elaborato sia nel percorso batch che in quello real-time, portando a una duplicazione degli sforzi.

2 Hadoop e MapReduce

2.1 Introduzione a Hadoop

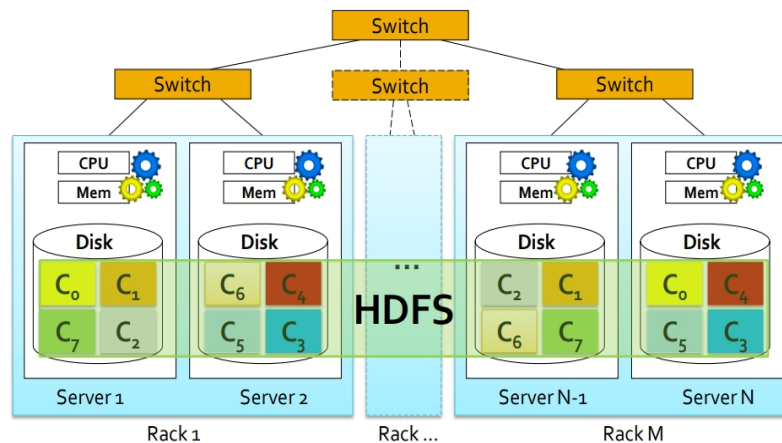
Apache **Hadoop** è un framework open-source progettato per l'elaborazione distribuita di grandi volumi di dati su cluster di computer. Utilizza un semplice modello di programmazione per consentire il calcolo scalabile ed efficiente su grandi quantità di dati, che possono essere strutturati o non strutturati. Hadoop è stato creato per risolvere i problemi legati all'elaborazione dei **Big Data**, garantendo affidabilità, tolleranza ai guasti e scalabilità.

L'architettura di Hadoop si basa su quattro componenti principali:

2.1.1 Hadoop Distributed File System (HDFS)

HDFS è un file system distribuito progettato per eseguire su hardware di fascia consumer. Consente la memorizzazione di dati su più nodi di un cluster e assicura l'accessibilità dei dati anche in caso di guasti hardware. Le sue principali caratteristiche includono:

- **Alta tolleranza ai guasti:** I dati vengono replicati su diversi nodi per garantire la disponibilità.
- **Scalabilità:** HDFS può scalare facilmente con l'aggiunta di nuovi nodi.
- **Gestione di grandi file:** È progettato per gestire file molto grandi, spesso di dimensioni pari a gigabyte o terabyte.



Anche se i dati sono distribuiti in nodi diversi è come se vedessimo un singolo livello costituito dall'HDFS. Questa struttura serve per distinguere cosa fare dal come è costruito. I programmatori possono focalizzarsi sul codice piuttosto che sul come è fatta l'intera struttura astruendo il concetto di file system distribuito. Se c'è una failure, il sistema distribuito riesce a gestire il problema in quanto distribuito. HDFS fornisce uno spazio dei nomi globale per i file, memorizza i dati in modo ridondante su più nodi per garantire persistenza e disponibilità. Sono presenti file enormi, i dati vengono raramente aggiornati e le operazioni di lettura e aggiunta sono comuni.

Inoltre, l'organizzazione dei file è fatta in questo modo:

- Ogni file è suddiviso in "chunk/blocchi" che sono distribuiti tra i server.
- Ogni chunk è replicato su server diversi (di solito ci sono 3 repliche per chunk).
- Questo garantisce persistenza e disponibilità.
- Per aumentare la persistenza e la disponibilità, le repliche sono memorizzate in rack diversi, se possibile.
- Ogni chunk/blocco contiene una parte del contenuto di un singolo file.
- Non è possibile avere il contenuto di due file nello stesso chunk/blocco.
- Tipicamente ogni chunk è di 64-128MB.

2.1.2 MapReduce

MapReduce è un modello di programmazione utilizzato da Hadoop per elaborare grandi dataset in modo parallelo. Il processo è suddiviso in due fasi principali:

- **Map:** I dati vengono suddivisi in frammenti e mappati per ottenere una serie di coppie chiave-valore.
- **Reduce:** Le coppie chiave-valore vengono aggregate e ridotte per ottenere i risultati finali.

MapReduce distribuisce automaticamente i task sui nodi del cluster, rendendo l'elaborazione scalabile e tollerante ai guasti.

2.1.3 YARN (Yet Another Resource Negotiator)

YARN è il gestore delle risorse all'interno di Hadoop, responsabile dell'allocazione delle risorse nei nodi del cluster. YARN separa le funzionalità di gestione delle risorse e di elaborazione, migliorando l'efficienza dell'intero sistema. Esso gestisce:

- **Allocazione delle risorse:** Distribuisce la memoria e la capacità di elaborazione tra le applicazioni.
- **Gestione dei job:** Coordina l'esecuzione dei vari job distribuiti nel cluster.

2.1.4 Hadoop Common

Hadoop Common contiene le librerie e le utility necessarie per i moduli Hadoop, fornendo le basi su cui tutti i componenti di Hadoop possono funzionare correttamente. Include funzionalità come l'accesso ai file e l'autenticazione.

2.1.5 Vantaggi di Hadoop

- **Scalabilità orizzontale:** Hadoop può scalare facilmente aggiungendo nuovi nodi al cluster senza modificare la struttura dell'applicazione.
- **Tolleranza ai guasti:** Grazie alla replicazione dei dati su più nodi, Hadoop è in grado di gestire guasti hardware senza perdita di dati.

- **Elaborazione parallela:** Utilizzando MapReduce, Hadoop consente l'elaborazione parallela di grandi volumi di dati, riducendo significativamente i tempi di calcolo.
- **Economico:** È possibile utilizzare hardware commodity (a basso costo) per costruire i cluster Hadoop.

2.1.6 Limitazioni di Hadoop

- **Latenza elevata:** Hadoop è più efficiente per l'elaborazione batch, ma non è adatto a operazioni in tempo reale o a bassa latenza.
- **Non adatto a dati piccoli:** Hadoop è stato progettato per gestire grandi quantità di dati; elaborare piccoli dataset può risultare inefficiente.
- **Complessità:** Configurare e gestire un cluster Hadoop richiede una notevole competenza tecnica.

2.1.7 Conclusione

Hadoop è uno strumento potente e flessibile per l'elaborazione distribuita di Big Data. Grazie alla sua architettura scalabile e alla tolleranza ai guasti, è ampiamente utilizzato in molte applicazioni, dalle analisi di dati alle applicazioni di machine learning. Tuttavia, richiede un'attenta gestione e può non essere la soluzione ideale per tutti i tipi di workload, soprattutto per applicazioni che necessitano di tempi di risposta in tempo reale.

2.1.8 Esempio pratico di elaborazione distribuita con Hadoop e MapReduce

Il problema considerato è il conteggio delle occorrenze di ciascuna parola in un file di testo di grandi dimensioni.

- **Input:** un grande file di testo contenente parole.
- **Problema:** contare quante volte ciascuna parola distinta appare nel file.
- **Output:** una lista di coppie `<parola, numero>`, dove ogni coppia rappresenta la parola e il numero di occorrenze.

Case 1: File che rientra in memoria principale Se l'intero file può essere caricato nella memoria di un singolo nodo, il modo più efficiente di risolvere il problema è utilizzare un approccio tradizionale su un singolo nodo. In questo caso, i vantaggi dell'elaborazione distribuita vengono superati dai costi di complessità e di gestione del sistema distribuito, che introducono un overhead. Tuttavia, cosa succede se il file è troppo grande per essere gestito da un singolo nodo?

Case 2: File troppo grande per rientrare in memoria Quando il file è troppo grande per essere caricato nella memoria principale, diventa necessario adottare un approccio distribuito. Questo porta alla domanda di come suddividere il problema in sottocompiti quasi indipendenti che possono essere eseguiti in parallelo su un cluster di server. Qui entra in gioco Hadoop con il modello MapReduce.

Elaborazione parallela su ciascun server Il file viene suddiviso in "chunk" (porzioni di dati). Ogni server nel cluster elabora un chunk e conta il numero di occorrenze delle parole in

quella porzione specifica del file. Durante questa fase, ogni server agisce in maniera indipendente dagli altri e non è necessaria alcuna sincronizzazione. Il risultato di ogni server è una lista parziale di coppie `<parola, numero>`, dove ogni coppia rappresenta il conteggio delle parole per quel chunk.

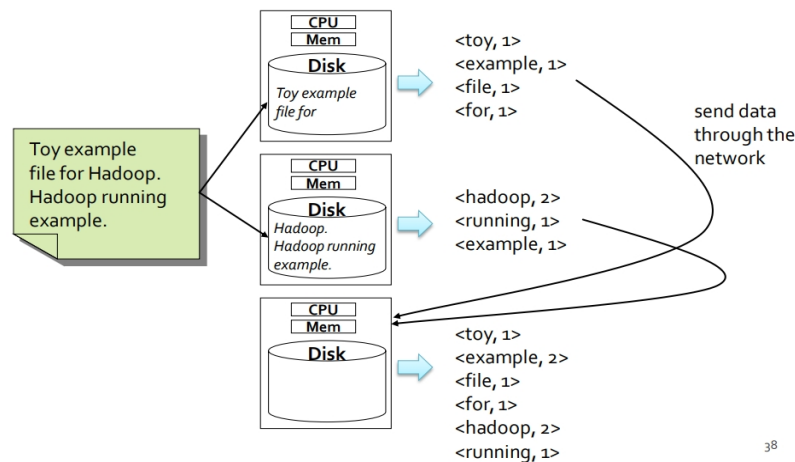
Esempio: Supponiamo di avere un file con il seguente contenuto:

"Toy example file for Hadoop. Hadoop running example."

Questo file viene suddiviso in due chunk. Uno dei server potrebbe produrre il seguente risultato:

```
<toy, 1> <example, 1> <file, 1> <for, 1> <hadoop, 2> <running, 1> <example, 1>
```

Ogni server elabora il suo chunk localmente, producendo risultati parziali senza la necessità di comunicazione immediata con gli altri server.



38

Aggregazione dei risultati parziali Dopo aver completato l'elaborazione locale, i server inviano le loro liste parziali al server di aggregazione, incaricato di calcolare il risultato globale. Questo server somma le occorrenze di ciascuna parola, provenienti dai vari chunk, e genera la lista globale di coppie `<parola, numero>`, che rappresenta il conteggio finale delle parole nel file. In questa fase, è necessaria una sincronizzazione tra i server.

2.1.9 Scalabilità

La scalabilità in un sistema distribuito come Hadoop/MapReduce può essere valutata lungo due dimensioni:

- **Scalabilità dei dati:** se la quantità di dati raddoppia, il tempo di esecuzione dovrebbe raddoppiare, poiché ciascun server processa il doppio dei dati.
- **Scalabilità delle risorse:** se raddoppiamo il numero di server, il tempo di esecuzione dovrebbe dimezzarsi, poiché ogni server processa metà dei dati.

2.1.10 Considerazioni su Hadoop/MapReduce

Hadoop e MapReduce sono progettati per l'elaborazione **batch** di grandi quantità di dati. Il loro punto di forza è l'elaborazione di applicazioni **data-intensive**, come:

- Analisi dell'intero web (ad esempio, il calcolo del **PageRank**).
- Analisi di grandi grafi sociali (come l'algoritmo di **Link Prediction** per i suggerimenti di amici).
- Analisi di grandi log, come tracce di rete o dati provenienti da smart meter.

Tuttavia, Hadoop/MapReduce non è la soluzione ideale per tutti i tipi di problemi. Non è adatto per problemi **iterativi** o **ricorsivi**, né per l'elaborazione di dati in streaming o per l'elaborazione in **tempo reale**, dove la latenza deve essere ridotta al minimo.

Hadoop, con il modello MapReduce, è un potente strumento per affrontare problemi di elaborazione su larga scala, in particolare per i **Big Data**. Grazie alla suddivisione dei dati e all'elaborazione parallela, è in grado di gestire file che non potrebbero essere trattati da un singolo server. Tuttavia, non è adatto per applicazioni che richiedono elaborazioni iterative, ricorsive o in tempo reale.

2.2 MapReduce su Java

Il framework **Hadoop MapReduce** è un modello di programmazione distribuita utilizzato per l'elaborazione di grandi dataset in modo parallelo su cluster di macchine. Il modello si basa sulla definizione di due funzioni principali: `map()` e `reduce()`. Gli sviluppatori si concentrano principalmente sulla definizione di queste funzioni, mentre Hadoop gestisce la distribuzione, l'esecuzione parallela e il coordinamento dei task distribuiti. In particolare, Hadoop si occupa dell'organizzazione delle fasi di *shuffle* e *sort* che intervengono tra la fase di mappatura e quella di riduzione.

2.2.1 Funzioni Map e Reduce

Le funzioni `map()` e `reduce()` rappresentano il cuore dell'elaborazione in MapReduce. La funzione `map()` riceve un input e lo trasforma in coppie chiave-valore. Durante questa fase, ogni Mapper elabora una porzione dei dati totali (detta *split*) e produce dati intermedi che vengono successivamente organizzati e passati ai Reducer.

La funzione `reduce()` invece prende come input una chiave e una lista di valori associati, e combina questi valori per produrre un output aggregato. Questa funzione è particolarmente utile quando si desidera ridurre una grande quantità di dati attraverso operazioni di somma, conteggio, media o altre funzioni di aggregazione.

L'esecuzione parallela di queste fasi consente a Hadoop di processare grandi volumi di dati in modo efficiente. Hadoop si occupa inoltre di gestire automaticamente la fase intermedia di *shuffle* (che raggruppa i dati per chiave) e di *sort* (che ordina i dati per chiave), oltre a coordinare l'assegnazione dei task ai diversi nodi del cluster.

2.2.2 Esecuzione Coordinata da Hadoop

Una delle principali caratteristiche di Hadoop è la gestione coordinata e distribuita delle fasi di Map e Reduce. Hadoop si occupa di diverse responsabilità che altrimenti graverebbero sugli sviluppatori.

Durante la fase di *shuffle* e *sort*, i dati prodotti dai Mapper vengono aggregati e ordinati per chiave. Ogni Reducer riceve un gruppo di chiavi e i relativi valori aggregati. Questa fase consente la distribuzione efficiente dei risultati intermedi e prepara i dati per la fase di riduzione.

La *pianificazione dei task* è un'altra funzione fondamentale del framework: Hadoop assegna i task ai vari nodi del cluster e si occupa di gestire eventuali errori di esecuzione (es. nodi che falliscono o non rispondono). Questo garantisce la resilienza e la tolleranza ai guasti.

2.2.3 Componenti del Programma Hadoop MapReduce

Un programma MapReduce in Hadoop è strutturato in tre componenti principali. La prima è il **Driver**, che gestisce la configurazione del lavoro e il coordinamento del flusso di esecuzione. Il Driver configura le classi Mapper e Reducer, specifica il formato di input e output, e imposta il numero di Reducer da utilizzare.

Il secondo componente è il **Mapper**, che esegue la fase di mappatura dei dati. Estendendo la classe base **Mapper**, l'utente può definire la logica personalizzata per trasformare i dati di input in coppie chiave-valore.

Il terzo componente è il **Reducer**, che esegue la fase di riduzione, combinando i valori associati a ciascuna chiave.

2.2.4 Esecuzione di Job e Task

Un *job* rappresenta l'esecuzione di un programma MapReduce su un dataset. Ogni job consiste di uno o più task, che sono unità esecutive di lavoro distribuito. I task Mapper e Reducer sono eseguiti sui dati distribuiti tra i vari nodi. Ogni Mapper elabora un *input split*, che è una porzione del dataset di input. Una volta che i Mapper hanno prodotto le coppie chiave-valore intermedie, la fase di *shuffle* distribuisce i dati ai Reducer, i quali aggregano i risultati finali per chiave.

2.2.5 Fasi di MapReduce

Le fasi principali dell'esecuzione di un job MapReduce possono essere suddivise in tre:

Fase Map Durante questa fase, la funzione `map()` riceve un input e lo trasforma in coppie chiave-valore. I dati elaborati dai Mapper sono temporaneamente memorizzati nel file system locale di ciascun nodo.

Fase Shuffle e Sort Questa fase è gestita internamente da Hadoop e non necessita di interventi manuali. I dati intermedi prodotti dai Mapper vengono raggruppati per chiave e ordinati. La fase *shuffle* distribuisce i dati dai Mapper ai Reducer, mentre la fase *sort* si occupa di ordinare le coppie chiave-valore.

Fase Reduce Durante la fase di riduzione, la funzione `reduce()` prende come input le copie chiave-lista di valori e le processa per produrre il risultato finale, che viene successivamente memorizzato su HDFS (Hadoop Distributed File System).

2.2.6 Distribuzione e Parallelizzazione

La distribuzione e la parallelizzazione sono due pilastri fondamentali di Hadoop. Ogni file di input viene suddiviso in più *input split*, ognuno dei quali viene elaborato in parallelo da un Mapper su uno o più nodi del cluster. Allo stesso modo, i Reducer sono distribuiti su più nodi, e ciascuno di essi riceve una parte dei dati raggruppati per chiave.

Hadoop sfrutta un'architettura di calcolo distribuito basata su nodi, che consente di scalare orizzontalmente il sistema aumentando il numero di nodi nel cluster. Ciò consente di processare dataset di dimensioni molto grandi in modo efficiente.

2.3 Programmare MapReduce in Java

L'idea di ora è come implementare ciò su Java. Un programma MapReduce consiste nell'implementare 3 classi:

- **Driver class:** Contiene il nome del lavoro, l'input, l'output, il nome delle classi implementate. In pratica contiene l'implementazione e le coordinate del progetto.
- **Mapper class:** Una classe che implementa la funzione di Map
- **Reducer class:** Una classe che implementa la funzione di Reduce

Poi abbiamo Driver, Mapper, Reducer che saranno le istanze delle classi (gli oggetti). Possiamo avere più istanze per parallelizzare le operazioni di map e reduce. Definiamo lavoro (Hadoop Job) è l'esecuzione di un programma MapReduce su un dataset, task è l'esecuzione di un'istanza di Mapper o Reducer su una parte di dati. I dati input sono divisi in gruppi, della stessa dimensione dei chunks nell'HDFS.

2.3.1 Tipi di Dati in Hadoop

Hadoop ha il proprio set di tipi di dati di base ottimizzati per la serializzazione in rete, utili nell'ambito del framework MapReduce. Di seguito, illustriamo ciascun concetto con degli esempi. In Hadoop, i tipi di dati di base sono:

- **Text:** simile a una stringa in Java (**String**), rappresenta una sequenza di caratteri.
- **IntWritable:** simile a **Integer** di Java, rappresenta un numero intero.
- **LongWritable:** simile a **Long** di Java, rappresenta un numero intero lungo.
- **FloatWritable:** simile a **Float** di Java, rappresenta un numero in virgola mobile.

Questi tipi sono ottimizzati per la serializzazione, poiché devono essere trasmessi tra i nodi durante l'esecuzione dei job di MapReduce.

Writable e WritableComparable

I tipi di dati di base implementano due interfacce fondamentali:

- **Writable:** si occupa della serializzazione/deserializzazione.
- **WritableComparable:** aggiunge la capacità di confrontare i dati (necessario per la fase di ordinamento).

Ad esempio:

- Le chiavi nel processo MapReduce devono essere confrontabili, per cui sono istanze di **WritableComparable** (ad esempio, un **IntWritable** può essere confrontato con un altro **IntWritable**).
- I valori possono essere solo istanze di **Writable** (anche se spesso implementano anche **WritableComparable**).

Creazione di Nuovi Tipi di Dati

Gli sviluppatori possono creare nuovi tipi di dati complessi implementando le interfacce `Writable` e/o `WritableComparable`. Questo è utile per gestire strutture dati personalizzate.

Il processo MapReduce riceve in input un file HDFS, che viene convertito in una serie di coppie (`key`, `value`). Il `Mapper` lavora con queste coppie per produrre nuovi output.

2.3.2 InputFormat

La classe `InputFormat` è responsabile della gestione dell'input:

- Convalida il formato del file di input.
- Divide il file in sezioni logiche, chiamate *input splits*.
- Fornisce il `RecordReader` per convertire ogni input split in coppie (`key`, `value`).

Esempi di InputFormat Predefiniti

TextInputFormat

`TextInputFormat` gestisce file di testo semplici, suddividendoli in righe. Ogni riga del file diventa una coppia (`key`, `value`). La chiave è la posizione della riga nel file (*offset*), mentre il valore è il contenuto della riga.

File di input HDFS:

```
0: "Toy example file for Hadoop."  
31: "Hadoop running example."  
56: "TextInputFormat is used to split data."
```

Risultato in coppie (`key`, `value`):

```
(0, "Toy example file for Hadoop.")  
(31, "Hadoop running example.")  
(56, "TextInputFormat is used to split data.")
```

KeyValueTextInputFormat

`KeyValueTextInputFormat` gestisce file di testo in cui ogni riga è composta da una chiave e un valore separati da un delimitatore (di default il tab `\t`). Il testo prima del delimitatore è la chiave, e il testo dopo il delimitatore è il valore.

File di input HDFS:

```
10125\tMister John  
10236\tMiss Jenny  
1\tMister Donald Duck
```

Risultato in coppie (`key`, `value`):

```
(10125, "Mister John")  
(10236, "Miss Jenny")  
(1, "Mister Donald Duck")
```

2.3.3 OutputFormat

La classe `OutputFormat` scrive il risultato del job MapReduce in HDFS. Una delle implementazioni più comuni è la `TextOutputFormat`, che scrive file di testo semplici in cui ogni coppia (`key`, `value`) è scritta come una riga nel file di output, con chiave e valore separati da un tab (`\t`).

Se le coppie (`key`, `value`) prodotte sono:

```
(10125, "Mister John")
(10236, "Miss Jenny")
(1, "Mister Donald Duck")
```

L'output sarà:

```
10125\tMister John
10236\tMiss Jenny
1\tMister Donald Duck
```

2.3.4 Driver

Il **Driver** è responsabile della configurazione del job. Esso imposta il Mapper e il Reducer, specifica il formato di input e output, e definisce il numero di Reducer da utilizzare.

```
1  /* Set package */
2  package it.polito.bigdata.hadoop.wordcount;
3  /* Import libraries */
4  import org.apache.hadoop.conf.Configuration;
5  import org.apache.hadoop.conf.Configured;
6  import org.apache.hadoop.fs.Path;
7  import org.apache.hadoop.io.IntWritable;
8  import org.apache.hadoop.io.Text;
9  import org.apache.hadoop.mapreduce.Job;
10 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
11 import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
12 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
13 import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
14 import org.apache.hadoop.util.Tool;
15 import org.apache.hadoop.util.ToolRunner;
16
17 public class WordCountDriver extends Configured implements Tool {
18     public int run(String[] args) throws Exception {
19         Job job = Job.getInstance(getConf(), "wordcount");
20         job.setJarByClass(WordCountDriver.class);
21
22         job.setMapperClass(WordCountMapper.class);
23         job.setReducerClass(WordCountReducer.class);
24
25         job.setOutputKeyClass(Text.class);
26         job.setOutputValueClass(IntWritable.class);
27
28         FileInputFormat.addInputPath(job, new Path(args[0]));
29         FileOutputFormat.setOutputPath(job, new Path(args[1]));
30
31         return job.waitForCompletion(true) ? 0 : 1;
32     }
33     /* main method of the driver class */
34     public static void main(String args[]) throws Exception {
```

```

35     /* Exploit the ToolRunner class to "configure" and run the
36     Hadoop application */
37
38     int res = ToolRunner.run(new Configuration(), new WordCount(), args);
39     System.exit(res);
40 } // End of the main method
41 }
42 } // End of public class WordCount

```

2.3.5 Mapper

Il **Mapper** prende in input ogni riga del file di testo, la suddivide in parole e emette una coppia chiave-valore per ciascuna parola, dove la chiave è la parola stessa e il valore è il numero 1.

```

1  /* Set package */
2  package it.polito.bigdata.hadoop.wordcount;
3  /* Import libraries */
4  import java.io.IOException;
5  import org.apache.hadoop.io.IntWritable;
6  import org.apache.hadoop.io.LongWritable;
7  import org.apache.hadoop.io.Text;
8  import org.apache.hadoop.mapreduce.Mapper;
9
10 public class WordCountMapper extends Mapper<LongWritable, Text, Text, IntWritable>
11 {
12     private final static IntWritable one = new IntWritable(1);
13     private Text word = new Text();
14
15     public void map(LongWritable key, Text value, Context context) throws
16     IOException, InterruptedException {
17         String[] words = value.toString().split("\\s+");
18         for (String str : words) {
19             word.set(str);
20             context.write(word, one);
21         }
22     }
23 }

```

In Hadoop, il parametro ‘Context’ nel metodo ‘map’ di un mapper è un oggetto che fornisce una connessione tra il framework Hadoop e il codice del mapper. È una parte fondamentale del meccanismo di mappatura e riduzione, e ha diverse funzioni importanti:

- **Comunicazione con il framework:** Il ‘Context’ permette al mapper di comunicare con il framework Hadoop, ad esempio per scrivere coppie chiave-valore di output attraverso il metodo ‘context.write(outputKey, outputValue)’.
- **Configurazione e contatori:** Il ‘Context’ fornisce l’accesso alla configurazione del job e ai contatori che possono essere utilizzati per tracciare statistiche o informazioni di debug.
- **Monitoraggio dello stato:** Si può aggiornare lo stato del mapper tramite ‘context.setStatus()’, che consente al framework di monitorare il progresso e l’attività del job.
- **Accesso ai dati:** Il ‘Context’ fornisce accesso ai dati di input, e permette al mapper di leggere e gestire le coppie (key, value) che arrivano dall’input.

Ecco un esempio di utilizzo tipico:

```

1 protected void map(LongWritable key, Text value, Context context)
2 throws IOException, InterruptedException {
3     String line = value.toString();
4     String[] words = line.split(" ");
5     for (String word : words) {
6         context.write(new Text(word), new IntWritable(1));
7     }
8 }

```

In questo esempio, ogni parola viene emessa come chiave (key) e il valore associato è 1 (output value). L'oggetto 'Context' è utilizzato per emettere questi risultati.

2.3.6 Reducer

Il **Reducer** prende in input una chiave (la parola) e una lista di valori (il numero di occorrenze). Somma i valori per ogni chiave e genera il conteggio totale per ciascuna parola.

```

1  /* Set package */
2  package it.polito.bigdata.hadoop.wordcount;
3  /* Import libraries */
4  import java.io.IOException;
5  import org.apache.hadoop.io.IntWritable;
6  import org.apache.hadoop.io.Text;
7  import org.apache.hadoop.mapreduce.Reducer;
8  public class WordCountReducer extends Reducer<Text, IntWritable, Text, IntWritable>
9  {
10     public void reduce(Text key, Iterable<IntWritable> values, Context context)
11     throws IOException, InterruptedException {
12         int sum = 0;
13         for (IntWritable val : values) {
14             sum += val.get();
15         }
16         context.write(key, new IntWritable(sum));
17     }
18 }

```

2.4 Combiner in Hadoop

In Hadoop, è possibile utilizzare un componente chiamato **Combiner** come pre-aggregatore prima della fase di *Reduce*. Il Combiner può essere considerato un *pre-Reducer*, in quanto combina i risultati prodotti dai *Mapper* prima di passarli alla fase di *Reduce*.

La classe che implementa un Combiner estende l'interfaccia del *Reducer*, poiché presenta gli stessi metodi, come `reduce()`. È importante notare che l'esecuzione del Combiner non è garantita: la decisione viene presa a runtime da Hadoop.

2.4.1 Funzione reduce()

La funzione `reduce()` del Combiner prende in input coppie formate da una chiave e una lista di valori, e restituisce una lista di coppie chiave-valore. Dopo aver applicato il Combiner, i risultati vengono passati al *Reducer*, che aggrega i dati presi dagli altri nodi.

2.4.2 Esempio di Combiner in Hadoop

Di seguito è riportato un esempio di implementazione di un Combiner:

```
1 import org.apache.hadoop.io.IntWritable;
2 import org.apache.hadoop.io.Text;
3 import org.apache.hadoop.mapreduce.Reducer;
4 import java.io.IOException;
5
6 public class ExampleCombiner extends Reducer<Text, IntWritable, Text, IntWritable> {
7
8     @Override
9     protected void reduce(Text key, Iterable<IntWritable> values, Context context)
10         throws IOException, InterruptedException {
11         int sum = 0;
12         for (IntWritable val : values) {
13             sum += val.get();
14         }
15         context.write(key, new IntWritable(sum));
16     }
17 }
```

Listing 1: Implementazione di un Combiner in Hadoop

2.5 Tipi di Dati Personalizzati

In Hadoop, spesso è necessario lavorare con dati complessi composti da più elementi, come contatori o somme. In questi casi, si possono definire **tipi di dati personalizzati**, che estendono l'interfaccia `Writable` di Hadoop.

2.5.1 Implementazione di Writable

Per implementare un tipo di dato personalizzato, è necessario definire i metodi `readFields(DataInput in)` per la deserializzazione e `write(DataOutput out)` per la serializzazione. Di solito, si ridefinisce anche il metodo `toString()` per formattare l'output in modo leggibile.

2.5.2 Esempio: Classe SumAndCountWritable

```
1 import java.io.DataInput;
2 import java.io.DataOutput;
3 import java.io.IOException;
4 import org.apache.hadoop.io.Writable;
5
6 public class SumAndCountWritable implements Writable {
7     private float sum = 0;
8     private int count = 0;
9
10    public float getSum() {
11        return sum;
12    }
13
14    public void setSum(float sumValue) {
15        this.sum = sumValue;
16    }
17 }
```

```
18 public int getCount() {
19     return count;
20 }
21
22 public void setCount(int countValue) {
23     this.count = countValue;
24 }
25
26 @Override
27 public void write(DataOutput out) throws IOException {
28     out.writeFloat(sum);
29     out.writeInt(count);
30 }
31
32 @Override
33 public void readFields(DataInput in) throws IOException {
34     sum = in.readFloat();
35     count = in.readInt();
36 }
37
38 @Override
39 public String toString() {
40     return "sum=" + sum + ", count=" + count;
41 }
42 }
```

Listing 2: Implementazione della classe SumAndCountWritable

2.6 Configurazione di Hadoop

Un elemento fondamentale in Hadoop è l'oggetto `Configuration`, utilizzato per condividere i parametri di configurazione tra le componenti dell'applicazione, come il driver, i mapper e i reducer.

2.6.1 Personalizzazione della Configurazione

È possibile impostare coppie di valori (nome, valore) tramite il metodo `set()` nel driver e recuperare tali parametri nei mapper o nei reducer tramite `getConfiguration().get()`.

```
1 Configuration conf = new Configuration();
2 conf.set("custom.parameter", "value");
3
4 // Recupero del valore nel mapper o reducer
5 String value = context.getConfiguration().get("custom.parameter");
```

Listing 3: Esempio di utilizzo dell'oggetto Configuration

2.7 Contatori Personalizzati

Hadoop fornisce una serie di **contatori predefiniti** che monitorano statistiche come il numero di record in input/output o il numero di byte trasmessi. È anche possibile definire **contatori personalizzati** utilizzando gli enum di Java.

2.7.1 Esempio di Contatore Personalizzato

```

1 public static enum CUSTOM_COUNTERS {
2     ERROR_COUNT,
3     MISSING_FIELDS
4 }
5
6 // Incremento del contatore
7 context.getCounter(CUSTOM_COUNTERS.ERROR_COUNT).increment(1);

```

Listing 4: Definizione di contatori personalizzati con enum

2.8 Map-Only Jobs

In alcune applicazioni, tutto il lavoro può essere svolto dai **Mapper** senza necessità di una fase di **Reduce**. Questi sono chiamati **Map-Only Jobs**. In questi casi, si imposta il numero di reducer a zero.

```

1 Job job = Job.getInstance(conf, "Map-Only Job");
2 job.setNumReduceTasks(0);

```

Listing 5: Impostazione di un Map-Only Job

2.9 In-Mapper Combiner

Gli **In-Mapper Combiner** migliorano l'efficienza aggregando i risultati localmente nel **Mapper**. Le variabili vengono inizializzate nel metodo **setup()** e aggiornate in **map()**. L'emissione delle coppie avviene nel metodo **cleanup()**.

```

1 public class InMapperCombiner extends Mapper<LongWritable, Text, Text, IntWritable>
2 {
3     private Map<String, Integer> inMapperMap;
4
5     @Override
6     protected void setup(Context context) throws IOException, InterruptedException {
7         inMapperMap = new HashMap<>();
8     }
9
10    @Override
11    protected void map(LongWritable key, Text value, Context context) throws
12        IOException, InterruptedException {
13        String word = value.toString();
14        inMapperMap.put(word, inMapperMap.getOrDefault(word, 0) + 1);
15    }
16
17    @Override
18    protected void cleanup(Context context) throws IOException, InterruptedException
19    {
20        for (Map.Entry<String, Integer> entry : inMapperMap.entrySet()) {
21            context.write(new Text(entry.getKey()), new IntWritable(entry.getValue()));
22        }
23    }
24 }

```

Listing 6: Esempio di In-Mapper Combiner

3 Lettura di Dati da Dataset Multipli in Hadoop

- **Contesto:** Alcune applicazioni richiedono di processare dati provenienti da più dataset, che possono avere formati diversi.
- **Gestione in Hadoop:** Hadoop consente di leggere da input multipli, supportando diversi formati per ciascun dataset grazie alla classe `MultipleInputs`.
- **Specificità dei Mapper:** È necessario definire un mapper specifico per ogni dataset in input, assicurandosi che le coppie chiave-valore emesse dai vari mapper abbiano tipi di dato coerenti.

3.1 Esempio di Applicazione con Dati da Sensori Diversi

- **Dati da Sensori:** Un caso di utilizzo comune è la raccolta di dati provenienti da sensori, dove ciascun sensore registra la stessa misura.
- **Formati Diversi:** I dati raccolti possono avere formati differenti, specialmente se i sensori sono stati sviluppati da produttori diversi.

3.2 Configurazione degli Input Multipli nel Driver

- **Uso di `addInputPath`:** Nel driver dell'applicazione, per configurare più dataset di input, si utilizza il metodo `addInputPath` della classe `MultipleInputs`:
 - Aggiunge un percorso di input alla volta.
 - Permette di specificare la classe di formato di input per ogni percorso.
 - Definisce la classe Mapper associata a ciascun percorso di input.

Esempio di Configurazione:

```
1 MultipleInputs.addInputPath(job, new Path(args[1]), TextInputFormat.class, Mapper1.class);
2 MultipleInputs.addInputPath(job, new Path(args[2]), TextInputFormat.class, Mapper2.class);
```

- Il codice sopra definisce due percorsi di input (`args[1]` e `args[2]`) utilizzando il formato `TextInputFormat`.
- `Mapper1` è usato per il primo percorso, e `Mapper2` per il secondo.

3.3 Distributed Cache in Hadoop

- **Necessità di File Condivisi:** Alcune applicazioni devono accedere a piccoli file in modalità di sola lettura per eseguire correttamente i compiti, distribuendo questi file in modo efficiente su tutti i nodi del cluster.
- **Vantaggio della Cache Distribuita:** La `DistributedCache` di Hadoop permette di mantenere copie dei file condivisi (presenti in HDFS) localmente su ciascun nodo, migliorando l'efficienza.

- Evita la lettura continua dal file system HDFS, poiché ogni nodo mantiene una copia locale dei file.
- I file condivisi possono includere testi, archivi o librerie `.jar` necessari all'applicazione.

3.4 Utilizzo del Distributed Cache

- **Aggiunta al Driver:** Nel driver, i file da condividere tramite cache distribuita vengono specificati con il metodo `job.addCacheFile(path)`.
- **Creazione di Copie Locali:** All'avvio del job, Hadoop crea una copia locale di ciascun file su ogni nodo che eseguirà task di mappatura o riduzione.
- **Lettura nei Mapper/Riduttori:** Il file cache viene letto localmente, tipicamente nel metodo `setup` del mapper o riduttore, consentendo un accesso più rapido rispetto alla lettura diretta da HDFS.

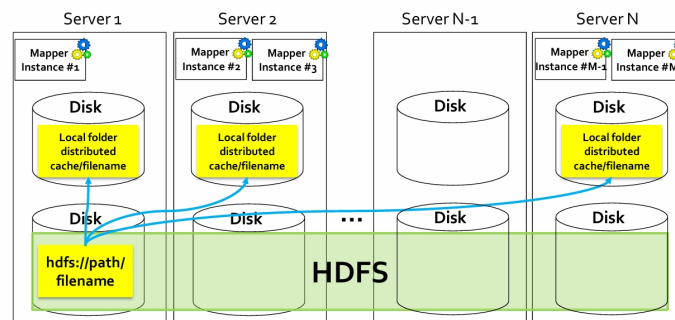


Figura 1: Distributed Cache

3.5 Efficienza e Utilizzo della Cache Distribuita

- **Efficienza Dipendente dal Carico:** L'efficienza della Distributed Cache aumenta se più mapper/riduttori condividono la stessa copia locale del file, riducendo la necessità di accedere al file system distribuito (HDFS).
- **Senza Cache Distribuita:** Senza il supporto della cache, ogni mapper o riduttore dovrebbe leggere il file direttamente da HDFS, il che rallenta l'accesso rispetto alla lettura dalla copia locale.

3.6 Esempio di Codice per la Gestione della Distributed Cache

Aggiunta del File alla Cache:

```
1 public int run(String[] args) throws Exception {
2     job.addCacheFile(new Path("hdfs path/filename").toUri());
3 }
```

Lettura del File in Cache nel Mapper:

```
1 protected void setup(Context context) throws IOException, InterruptedException {  
2     URI[] urisCachedFiles = context.getCacheFiles();  
3     BufferedReader file = new BufferedReader(new FileReader(new File(new Path(  
4         urisCachedFiles[0].getPath()).getName())));  
5     String line;  
6     while ((line = file.readLine()) != null) {  
7         // processa la linea  
8     }  
9     file.close();  
}
```

- **Note:**

- Il file in cache viene salvato in una cartella locale temporanea su ciascun server utilizzato per l'applicazione.
- Questa cartella è separata dal percorso originale del file in HDFS.

4 Introduzione ai Pattern di MapReduce

I pattern di MapReduce sono utilizzati per implementare applicazioni che generano una vista riassuntiva e di alto livello dei dati, come la produzione di statistiche numeriche o l'indicizzazione dei dati. Lo scopo è facilitare l'analisi delle informazioni principali, rendendole facilmente accessibili e interpretabili.

4.1 Sommatorie Numeriche

L'obiettivo delle sommatorie numeriche è raggruppare i record o gli oggetti in base a uno o più campi chiave e calcolare una statistica aggregata (ad esempio, media, massimo, minimo, deviazione standard) per ciascun gruppo.

Esempio: Supponiamo di avere una grande mole di dati sulle vendite. Utilizzando MapReduce, possiamo raggruppare le vendite per prodotto e calcolare la media o la somma delle vendite per ciascun prodotto, ottenendo così una visione globale dell'andamento delle vendite.

Funzionamento: I **mapper** emettono coppie chiave-valore (chiave, valore), dove la chiave è il campo usato per i gruppi e il valore contiene i dati utilizzati per calcolare le statistiche. I **reducer** raccolgono i valori per ogni chiave e calcolano il risultato finale per ciascun gruppo. Se la statistica è commutativa e associativa, i **combiner** possono aggregare i risultati parziali direttamente nei mapper per ridurre i tempi di calcolo.

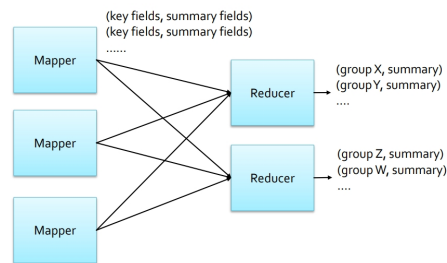


Figura 2: Sommatorie numeriche

4.2 Indice Invertito

L'indice invertito costruisce un indice dai dati di input per supportare una ricerca più rapida o un arricchimento dei dati.

Esempio: Immaginiamo di avere molti documenti di testo e di voler creare un indice delle parole. Con MapReduce, possiamo associare ogni parola (chiave) a una lista di identificatori di documenti (valori) in cui appare. Questo è alla base dei motori di ricerca: ogni parola è una chiave e i valori sono gli URL delle pagine che contengono quella parola.

Funzionamento: I **mapper** emettono una coppia (parola, ID documento) per ciascun termine, e i **reducer** raccolgono questi ID per ciascuna parola, concatenandoli in una lista.

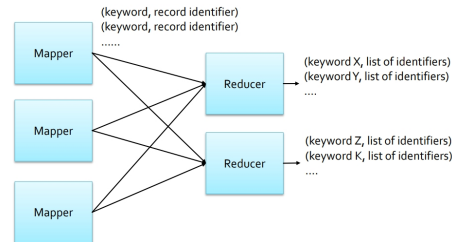


Figura 3: Indice Invertito

4.3 Filtraggio

Il filtraggio seleziona solo i record di interesse, scartando quelli che non soddisfano specifici criteri.

Esempio: Supponiamo di avere un dataset di transazioni e di voler analizzare solo quelle sopra una certa soglia. In questo caso, il mapper restituisce solo i record che soddisfano la condizione, ad esempio transazioni con importi maggiori di 100?. La riduzione è inutile, quindi questa operazione è chiamata *job di sola mappa*.

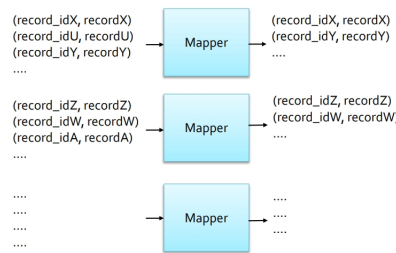


Figura 4: Filtraggio

4.4 Top K

Il pattern Top K seleziona un gruppo di record di maggior interesse in base a una funzione di ranking.

Esempio: Se vogliamo individuare i 10 prodotti più venduti, ogni mapper mantiene una lista dei primi K prodotti locali. Alla fine, ciascun mapper invia i suoi Top K record a un unico reducer, che li unisce e seleziona i primi 10 globali.

Funzionamento: I **mapper** mantengono un elenco dei Top K record locali, aggiornato con ogni nuovo record. Un singolo **reducer** aggrega questi risultati locali per creare il Top K finale.

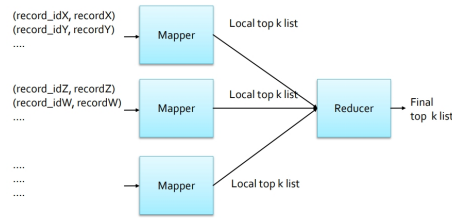


Figura 5: Top K

4.5 Selezione di Valori Distinti

Quando si desidera ottenere un insieme unico di valori, il pattern di selezione dei valori distinti è utile per eliminare duplicati.

Esempio: Supponiamo di avere un dataset con record di clienti duplicati. I mapper emettono una coppia (record, null) per ogni record di input, e i reducer emettono una sola copia per ogni record distinto, rimuovendo i duplicati.

Funzionamento: I **mapper** emettono una coppia per ogni record di input, mentre i **reducer** restituiscono una sola copia per ciascun record, eliminando i duplicati.

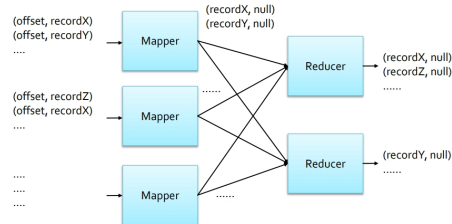


Figura 6: Selezione di valori distinti

4.6 Shuffle (Riordino Casuale)

Il riordino casuale dei dati viene utilizzato per anonimizzare o selezionare un sottoinsieme casuale di record.

Esempio: Un dataset di utenti può essere rimescolato per anonimizzare i dati o selezionare un campione casuale. I mapper emettono una coppia (chiave casuale, record) per ogni record, e i reducer ordinano casualmente i dati in uscita.

4.7 Job Chaining

Il job chaining orchestra una sequenza di lavori che compongono applicazioni complesse basate su molte fasi.

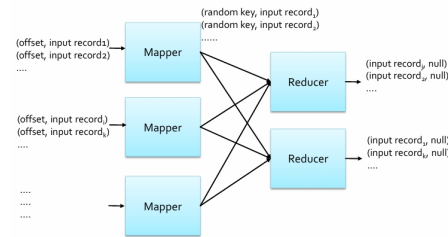


Figura 7: Shuffle

Esempio: In un'applicazione complessa di analisi dati, un job iniziale può filtrare i record indesiderati, seguito da un job per calcolare la media per gruppi e un job finale che seleziona il Top K delle anomalie rilevanti.

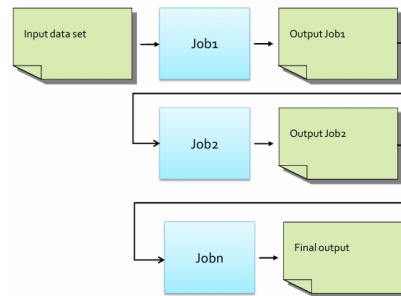


Figura 8: Job Chaining

Funzionamento: Il **driver** gestisce il workflow, eseguendo i job in ordine. I **mappers**, **reducers**, e **combiners** di ciascuna fase si occupano delle operazioni specifiche.

4.8 Join

Il pattern join combina due relazioni in base a un attributo comune, come un database relazionale.

4.8.1 Esempio di join lato reduce:

Per unire una tabella *Utenti* e una tabella *Preferenze*, i mapper emettono una coppia con l'attributo comune (ad esempio, l'ID utente) e le informazioni della tabella corrente. I reducer raccolgono le coppie per ciascun ID e le combinano.

4.8.2 Esempio di join lato map:

Quando una tabella è piccola, come un elenco di codici di categoria, può essere caricata nella cache distribuita per permettere ai mapper di unirla localmente ai record della tabella grande.

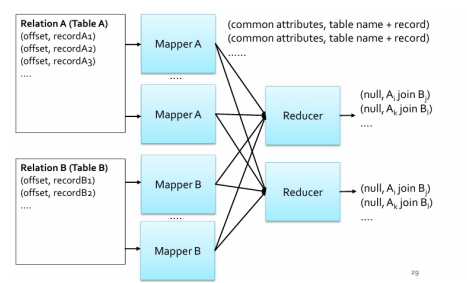


Figura 9: Join

5 Introduzione ad Apache Spark

Apache Spark è un motore di elaborazione per Big Data, rapido e generico, che consente di processare grandi quantità di dati in modo distribuito. Originariamente sviluppato presso l'AMPLab dell'Università della California - Berkeley, Spark ha come obiettivi principali:

- **Generalità:** supporta diversi tipi di carichi di lavoro, operatori e dimensioni di lavoro.
- **Bassa latenza:** risponde entro pochi secondi.
- **Tolleranza ai guasti:** considera i guasti come normali, non eccezionali.
- **Semplicità:** spesso raggiunta grazie alla generalità.

5.1 Esecuzione Iterativa: Confronto tra MapReduce e Spark

5.2 Problema con MapReduce

Nelle operazioni iterative, **MapReduce** richiede frequenti operazioni di I/O su disco, rallentando notevolmente l'esecuzione. Ogni fase (mapper, reducer) comporta lettura e scrittura su disco, anche per dati già processati.

5.3 Soluzione proposta da Spark

Spark conserva i dati in memoria principale per ridurre le operazioni di I/O. Ciò è possibile grazie alla riduzione dei costi della memoria principale, permettendo che grandi quantità di dati risiedano in RAM.

- **MapReduce:** leggere/scrivere da HDFS per ogni iterazione.
- **Spark:** leggere una sola volta i dati da HDFS, poi mantenerli in memoria.

Risultato: Spark è dalle 10 alle 100 volte più veloce rispetto a MapReduce grazie all'uso della memoria principale.

5.4 RDD (Resilient Distributed Dataset)

Gli **RDD** sono una rappresentazione distribuita di oggetti, immagazzinati principalmente in memoria o su disco locale, e consentono a Spark di eseguire operazioni parallele in modo efficiente.

5.5 Caratteristiche principali degli RDD

- **Immutabili:** una volta creati, gli RDD non possono essere modificati.
- **Tolleranza ai guasti:** Spark può ricostruire un RDD in caso di guasto usando un **grafo aciclico diretto** (DAG).
- **Suddivisione in partizioni:** ogni nodo del cluster contiene una partizione degli RDD, permettendo l'elaborazione parallela.

5.6 Creazione di un RDD

Gli RDD possono essere creati:

- Parallelizzando una collezione (es. lista in Python).
- Leggendo dati da HDFS o altri file system.
- Applicando trasformazioni su RDD esistenti.

5.7 Programmazione in Spark: Operazioni sugli RDD

Spark supporta due tipi di operazioni sugli RDD:

- **Trasformazioni:** map, filter, join, ecc.
- **Azioni:** count, collect, save, ecc.

Esempio di codice: conteggio righe di un file

```
1 from pyspark import SparkConf, SparkContext
2
3 if __name__ == "__main__":
4     conf = SparkConf().setAppName("Spark Line Count")
5     sc = SparkContext(conf=conf)
6     inputFile = "myfile.txt"
7
8     linesRDD = sc.textFile(inputFile)
9     numLines = linesRDD.count()
10    print("NumLines:", numLines)
11    sc.stop()
```

Listing 7: Conteggio righe in Spark

5.8 Componenti principali di Spark

Spark è composto da moduli che sfruttano il **Spark Core**:

- **Spark SQL:** per l'elaborazione di dati strutturati, supporta SQL e HiveQL.
- **Spark Streaming:** per processare flussi di dati in tempo reale.
- **MLlib:** libreria per machine learning e data mining.
- **GraphX:** libreria per la manipolazione di grafi.

5.9 Esempio di Word Count con Spark

Obiettivo: calcolare la frequenza delle parole in un file di testo e salvare i risultati in una cartella di output.

```
1 from pyspark import SparkConf, SparkContext
2 import sys
3
4 if __name__ == "__main__":
5     inputFile = sys.argv[1]
6     outputPath = sys.argv[2]
7
8     conf = SparkConf().setAppName("Spark Word Count")
9     sc = SparkContext(conf=conf)
10
11     lines = sc.textFile(inputFile)
12     words = lines.flatMap(lambda line: line.split())
13     words_one = words.map(lambda word: (word, 1))
14     counts = words_one.reduceByKey(lambda c1, c2: c1 + c2)
15
16     counts.saveAsTextFile(outputPath)
17     sc.stop()
```

Listing 8: Conteggio parole in Spark

5.10 Ecosistema Spark: Schedulazione e Parallelismo

Spark supporta diversi scheduler:

- **YARN**: scheduler standard di Hadoop.
- **Mesos**: popolare nel mondo del calcolo distribuito.
- **Standalone Scheduler**: scheduler semplice incluso in Spark.

Ogni applicazione Spark è composta da:

- **Driver program**: esegue la funzione principale e crea lo `SparkContext`.
- **Cluster manager**: servizio esterno per l'allocazione delle risorse.
- **Worker nodes**: nodi che eseguono il codice dell'applicazione.

6 RDD-Resilient Distributed Dataset

6.1 Spark Context

- La connessione del driver al cluster è basata sull'oggetto `Spark Context`.
- In Python, la classe si chiama `SparkContext`.
- Lo `Spark Context` si crea tramite il costruttore della classe `SparkContext`, il cui unico parametro è un oggetto di configurazione.

```
1 from pyspark import SparkConf, SparkContext
2
3 # Crea un oggetto di configurazione e imposta il nome dell'applicazione
4 conf = SparkConf().setAppName("Application name")
5
6 # Crea un oggetto Spark Context
7 sc = SparkContext(conf=conf)
```

- L'oggetto `SparkContext` può essere ottenuto anche tramite il metodo `SparkContext.getOrCreate(conf)`, che restituisce l'oggetto esistente (se già creato) o ne crea uno nuovo.
- C'è sempre un solo oggetto `SparkContext` per ogni applicazione.

6.2 RDD - Resilient Distributed Dataset

- Un RDD è una collezione distribuita e immutabile di oggetti.
- Ogni RDD è suddiviso in partizioni, permettendo l'esecuzione parallela del codice su ogni partizione in modo indipendente.
- Gli RDD possono contenere qualsiasi tipo di oggetto in Scala, Java o Python, comprese le classi definite dall'utente.

6.3 Creazione di un RDD

Gli RDD possono essere creati:

- Caricando un dataset esterno (ad esempio, il contenuto di una cartella o un singolo file).
- Parallelizzando una collezione locale di oggetti (ad esempio, una lista Python).

```
1 inputFile = "myfile.txt"
2 lines = sc.textFile(inputFile)
```

Nessuna computazione viene eseguita quando si invoca `sc.textFile()`: Spark registra solo come creare l'RDD e legge i dati in modo lazy solo quando necessario.

```
1 inputList = ['First element', 'Second element', 'Third element']
2 distRDDList = sc.parallelize(inputList)
```

Un RDD può essere salvato in file di testo distribuiti usando il metodo `saveAsTextFile(path)` dell'RDD.

```
1 outputPath = "risFolder/"
2 linesRDD.saveAsTextFile(outputPath)
```

Il metodo `collect()` permette di raccogliere i dati di un RDD nei nodi del cluster e di memorizzarli in una variabile Python locale.

```
1 contentOfLines = linesRDD.collect()
```

6.4 Tipi di Operazioni su un RDD

Gli RDD supportano due tipi principali di operazioni:

- **Transformations:** Operazioni che ritornano un nuovo RDD.
- **Actions:** Operazioni che ritornano risultati al driver o scrivono il risultato su disco.

Transformations

- Le transformations sono calcolate in modo lazy, ovvero vengono eseguite solo quando un'azione viene applicata.
- Spark costruisce un grafo delle dipendenze chiamato **lineage graph** per tracciare quali RDD sono stati utilizzati per crearne uno nuovo.

Actions Le actions sono operazioni che restituiscono risultati al programma Driver o salvano i risultati nello storage.

Vediamo ora un'esempio di codice in Python:

```
1 from pyspark import SparkConf, SparkContext
2
3 conf = SparkConf().setAppName("Spark Application")
4 sc = SparkContext(conf=conf)
5
6 # Legge il contenuto di un file di log
7 inputRDD = sc.textFile("log.txt")
8
9 # Filtra le righe contenenti "error"
10 errorsRDD = inputRDD.filter(lambda line: line.find('error') >= 0)
11
12 # Filtra le righe contenenti "warning"
13 warningRDD = inputRDD.filter(lambda line: line.find('warning') >= 0)
14
15 # Unisce gli errori e i warning in un nuovo RDD
16 badLinesRDD = errorsRDD.union(warningRDD)
17
18 # Rimuove i duplicati e conta le righe "problematiche"
19 uniqueBadLinesRDD = badLinesRDD.distinct()
20 numBadLines = uniqueBadLinesRDD.count()
21
22 print("Lines with problems:", numBadLines)
```

7 Spark in Python

7.1 Funzioni Lambda

Le slide descrivono come Spark gestisce le trasformazioni (e alcune azioni) su RDD (Resilient Distributed Dataset) utilizzando funzioni fornite dall'utente. Nello specifico, alcune operazioni in Spark richiedono che l'utente definisca funzioni che specificano quale tipo di "trasformazione" deve essere applicata agli elementi di un RDD. Ad esempio, la trasformazione `filter()` permette di selezionare solo gli elementi di un RDD che soddisfano una determinata condizione, specificata attraverso una funzione booleana fornita dall'utente. Questa funzione viene applicata a ciascun elemento dell'RDD di input e, se restituisce `True`, l'elemento viene incluso nel nuovo RDD; altrimenti, viene scartato.

Ogni linguaggio di programmazione ha il proprio metodo per passare funzioni alle trasformazioni e azioni di Spark. In **Python**, si possono utilizzare **lambda expressions** o **funzioni locali definite dall'utente**. Le lambda expressions permettono di scrivere funzioni semplici e concise, con una sola espressione, e sono molto comode per creare velocemente funzioni di filtro. Le funzioni locali definite (con la sintassi `def`) sono invece utilizzate per funzioni che richiedono più istruzioni o non restituiscono un valore. Le funzioni `def` risultano particolarmente utili quando è necessaria una struttura più complessa o si desidera riutilizzare la funzione in altre parti del codice.

Un esempio pratico mostra come creare un RDD a partire da un file di log e come applicare il filtro per selezionare solo le righe contenenti la parola "error". L'operazione di lettura di un file di log viene effettuata con `sc.textFile("log.txt")`, che crea un RDD con ciascuna riga del file come elemento dell'RDD stesso. Per filtrare le righe, si può utilizzare una **lambda function** come in `errorsRDD = inputRDD.filter(lambda l: l.find('error') >= 0)`. Questa espressione lambda verifica per ogni riga `l` se contiene il termine "error?". Se `find('error')` restituisce un valore maggiore o uguale a zero, significa che la parola è presente nella riga, e quindi la funzione restituisce `True`, facendo sì che la riga venga inclusa in `errorsRDD`.

Alternativamente, lo stesso filtro può essere implementato con una funzione definita dall'utente:

```
1 def myFunction(l):
2     return l.find('error') >= 0
3 inputRDD = sc.textFile("log.txt")
4 errorsRDD = inputRDD.filter(myFunction)
```

In questo caso, **myFunction** è una funzione booleana che accetta come input una riga `l` e verifica se contiene la parola "error", restituendo `True` o `False`. Questa soluzione è equivalente alla versione lambda, ma è più leggibile quando si hanno più condizioni o quando la funzione potrebbe essere riutilizzata.

Entrambi gli approcci sono simili in termini di efficienza, ma presentano vantaggi diversi. Il codice con lambda function è più compatto e leggibile, ma non è adatto per funzioni che richiedono più istruzioni o che non restituiscono un valore. Le funzioni definite localmente permettono invece una maggiore flessibilità e riusabilità, rendendole ideali per funzioni più articolate o utilizzabili in diversi contesti.

In conclusione, Spark permette di utilizzare diverse tipologie di funzioni per le trasformazioni e azioni sugli RDD, offrendo agli sviluppatori una notevole flessibilità per adattare le operazioni alle

specifiche esigenze dell'applicazione, sia che si tratti di filtri semplici tramite lambda expressions sia di logiche più complesse attraverso funzioni definite localmente.

7.2 Trasformazioni su Spark

Le operazioni di trasformazione servono ad analizzare un RDD e a ritornare un nuovo RDD con i dati elaborati. Vediamo ora alcune operazioni di trasformazione.

7.2.1 filter

La trasformazione `filter` permette di selezionare elementi di un RDD che soddisfano una condizione specifica. La sintassi `filter(f)` richiede che venga passata una funzione `f` che restituisce un valore booleano. Questa funzione viene applicata su ogni elemento `e` dell'RDD di input:

- Se `f(e)` è `True`, l'elemento `e` viene incluso nel nuovo RDD.
- Se `f(e)` è `False`, `e` viene scartato.

```
1 # Leggere le righe di un file di log in un RDD
2 inputRDD = sc.textFile("log.txt")
3 # Creare un nuovo RDD contenente solo le righe con la parola "error"
4 errorsRDD = inputRDD.filter(lambda e: e.find('error') >= 0)
```

Listing 9: Filtrare righe contenenti "error"

Con numeri, il filtro può essere utilizzato per selezionare valori superiori a un certo numero:

```
1 # Creare un RDD con i valori [1, 2, 3, 3]
2 inputList = [1, 2, 3, 3]
3 inputRDD = sc.parallelize(inputList)
4 # Selezionare solo quelli maggiori di 2
5 greaterRDD = inputRDD.filter(lambda num: num > 2)
```

Listing 10: Filtrare valori maggiori di 2

7.2.2 map

La trasformazione `map` è usata per trasformare ogni elemento dell'RDD di input in un nuovo valore specificato dalla funzione `f`. La nuova RDD contiene un elemento per ciascun elemento di input; il tipo può cambiare.

```
1 # Leggere un file contenente cognomi e creare un RDD con le lunghezze di ogni
   cognome
2 inputRDD = sc.textFile("usernames.txt")
3 lengthsRDD = inputRDD.map(lambda line: len(line))
```

Listing 11: Calcolare la lunghezza dei cognomi

```
1 # Calcolare il quadrato di ciascun numero in un RDD di interi
2 inputList = [1, 2, 3, 3]
3 inputRDD = sc.parallelize(inputList)
4 squaresRDD = inputRDD.map(lambda element: element * element)
```

Listing 12: Calcolare il quadrato di ciascun numero

7.2.3 flatMap

La trasformazione `flatMap` applica una funzione a ciascun elemento dell'RDD di input e genera una lista di valori. La nuova RDD contiene la concatenazione di tutte le liste risultanti.

```
1 # Leggere un file di testo e ottenere tutte le parole del documento in un nuovo RDD
2 inputRDD = sc.textFile("document.txt")
3 listOfWordsRDD = inputRDD.flatMap(lambda l: l.split(' '))
```

Listing 13: Creare una lista di parole

7.2.4 distinct

La trasformazione `distinct` rimuove i duplicati in un RDD. Per eseguire questa operazione, Spark usa un'operazione di shuffle per assicurare che gli elementi identici vengano raggruppati.

```
1 # Rimuovere duplicati da una lista di nomi
2 distinctNamesRDD = inputRDD.distinct()
```

Listing 14: Rimuovere duplicati da una lista di nomi

```
1 # Eliminare duplicati in un RDD di numeri [1, 2, 3, 3]
2 distinctIntRDD = inputRDD.distinct()
```

Listing 15: Eliminare duplicati in un RDD di numeri

7.2.5 sortBy

La trasformazione `sortBy` ordina gli elementi di un RDD in base a una chiave specificata dalla funzione `keyfunc`. Il parametro `ascending` permette di scegliere tra ordine crescente e decrescente.

```
1 # Ordinare un RDD contenente nomi in ordine alfabetico
2 inputRDD = sc.textFile("names.txt")
3 sortedNamesRDD = inputRDD.sortBy(lambda name: name)
```

Listing 16: Ordinare alfabeticamente un RDD di nomi

```
1 # Ordinare un RDD di nomi per lunghezza
2 sortedNamesLenRDD = inputRDD.sortBy(lambda name: len(name))
```

Listing 17: Ordinare un RDD di nomi per lunghezza

7.2.6 Introduzione alle trasformazioni tra due RDD

Spark fornisce una serie di trasformazioni che operano su due RDD di input e generano un nuovo RDD di output. Tra le principali trasformazioni su due RDD, vi sono:

- **Union:** unisce gli elementi di due RDD, includendo i duplicati.
- **Intersection:** restituisce solo gli elementi comuni a entrambi gli RDD, senza duplicati.
- **Subtract:** restituisce gli elementi presenti solo nel primo RDD (su cui è invocato il metodo) ma non nel secondo.

- **Cartesian:** calcola il prodotto cartesiano tra due RDD, creando coppie di ogni possibile combinazione.

Andando di più nel dettaglio a livello di codice:

- **Union:** La trasformazione `union` viene eseguita con il metodo `union(other)`, dove `other` è l'altro RDD. Questa operazione:
 - Restituisce un nuovo RDD contenente tutti gli elementi dei due RDD di input, inclusi i duplicati.
 - Non rimuove i duplicati per evitare operazioni di **shuffle** (costose in termini computazionali), dato che richiederebbe una visione globale di tutti gli elementi dei due RDD.

Se è necessario rimuovere i duplicati dopo una `union`, si può applicare la trasformazione `distinct()` sull'RDD risultante. Tuttavia, `distinct()` è costoso poiché implica uno shuffle.

- **Intersection:** La trasformazione `intersection` utilizza il metodo `intersection(other)`, dove `other` è il secondo RDD:
 - Restituisce un RDD contenente solo gli elementi presenti in entrambi gli RDD, senza duplicati.
 - Questa operazione implica uno **shuffle** poiché è necessario confrontare elementi tra le partizioni dei due RDD per trovare quelli comuni.
- **Subtract:** La trasformazione `subtract` utilizza il metodo `subtract(other)`, dove `other` è l'altro RDD:
 - Restituisce un RDD con gli elementi presenti solo nell'RDD su cui è invocato `subtract`.
 - Questa trasformazione mantiene eventuali duplicati presenti nel primo RDD.
 - Come `intersection`, anche `subtract` richiede uno shuffle per confrontare gli elementi tra le partizioni.
- **Cartesian:** La trasformazione `cartesian` usa il metodo `cartesian(other)` e:
 - Restituisce un nuovo RDD contenente tutte le possibili coppie di elementi tra il primo e il secondo RDD.
 - Non richiede che i due RDD abbiano lo stesso tipo di dato; permette quindi di eseguire il prodotto cartesiano tra RDD di tipi diversi.
 - Questa operazione può generare una grande quantità di dati poiché ogni elemento del primo RDD viene accoppiato con ogni elemento del secondo, rendendo la trasmissione di dati tra i nodi del cluster potenzialmente elevata.

Di seguito, alcuni esempi di codice per eseguire le operazioni di **union**, **intersection**, **subtract** e **cartesian**.

```

1 # Creare due RDD di interi
2 inputList1 = [1, 2, 2, 3, 3]
3 inputRDD1 = sc.parallelize(inputList1)
4 inputList2 = [3, 4, 5]
5 inputRDD2 = sc.parallelize(inputList2)
6
7 # Applicare le trasformazioni
8 outputUnionRDD = inputRDD1.union(inputRDD2)
9 outputIntersectionRDD = inputRDD1.intersection(inputRDD2)
10 outputSubtractRDD = inputRDD1.subtract(inputRDD2)

```

Listing 18: Union, Intersection e Subtract

```

1 # Prodotto cartesiano tra due RDD di interi
2 outputCartesianRDD = inputRDD1.cartesian(inputRDD2)

```

Listing 19: Cartesian

```

1 # Creare un RDD di interi e uno di stringhe
2 inputList1 = [1, 2, 3]
3 inputRDD1 = sc.parallelize(inputList1)
4 inputList2 = ["A", "B"]
5 inputRDD2 = sc.parallelize(inputList2)
6
7 # Prodotto cartesiano tra interi e stringhe
8 outputCartesianRDD = inputRDD1.cartesian(inputRDD2)

```

Listing 20: Cartesian con tipi misti

Questi esempi mostrano come creare nuovi RDD utilizzando le trasformazioni che operano su due RDD di input.

7.3 Azioni su Spark

Quando si applica un'azione si può memorizzare il risultato in una variabile locale del driver. La prima azione di cui parliamo è `collect()`. `Collect` ritorna una lista di oggetti locale contenente gli stessi oggetti del RDD considerato. Bisogna fare attenzione alla quantità occupata.

```

1 # Create an RDD of integers. Load the values 1, 2, 3, 3 in this RDD
2 inputList = [1, 2, 3, 3]
3 inputRDD = sc.parallelize(inputList)
4 # Retrieve the elements of the inputRDD and store them in
5 # a local python list
6 retrievedValues = inputRDD.collect()
7

```

Listing 21: Esempio collect

Altri metodi sono:

- **count**: ritorna il numero di oggetti dell'RDD.

```

1 # Read the content of the two input textual files
2 inputRDD1 = sc.textFile("document1.txt")
3 inputRDD2 = sc.textFile("document2.txt")
4 # Count the number of lines of the two files = number of elements
5 # of the two RDDs

```

```

6      numLinesDoc1 = inputRDD1.count()
7      numLinesDoc2 = inputRDD2.count()
8      if numLinesDoc1 > numLinesDoc2:
9          print("document1.txt")
10     elif numLinesDoc2 > numLinesDoc1:
11         print("document2.txt")
12     else:
13         print("Same number of lines")
14

```

Listing 22: Esempio count

- **countByValue()**: dato un RDD come input, ritorna una coppia per ogni oggetto contenente il nome e il numero dell'occorrenza.

```

1      # Read the content of the input textual file
2      namesRDD = sc.textFile("names.txt")
3      # Compute the number of occurrences of each name
4      namesOccurrences = namesRDD.countByValue()
5

```

Listing 23: Esempio countByValue

- **take(num)**: ritorna una lista locale di python di oggetti che contiene num oggetti dell'RDD considerato.

```

1      # Create an RDD of integers. Load the values 1, 5, 3, 3,2 in this RDD
2      inputList = [1, 5, 3, 3, 2]
3      inputRDD = sc.parallelize(inputList)
4      # Retrieve the first two elements of the inputRDD and store them in
5      # a local python list
6      retrievedValues = inputRDD.take(2)
7

```

Listing 24: Esempio take

- **first()**: ritorna un oggetto contenente il primo elemento dell'RDD. A differenza di take(1), first restituisce un oggetto mentre take(1) restituisce ugualmente una lista.
- **top(num)**: ritorna i num oggetti più grandi contenuti in una lista. Un'altra versione è top(num, key) dove key è una lambda function.

```

1      # Create an RDD of integers. Load the values 1, 5, 3, 4,2 in this RDD
2      inputList = [1, 5, 3, 4, 2]
3      inputRDD = sc.parallelize(inputList)
4      # Retrieve the top-2 elements of the inputRDD and store them in
5      # a local python list
6      retrievedValues = inputRDD.top(2)
7

```

Listing 25: Esempio top

- **takeOrdered(num)**: prende i num più piccoli dell'RDD e li mette in una lista. Anche in questo caso abbiamo la versione takeOrdered(num, key) in cui key è una lambda function.

```

1  # Create an RDD of integers. Load the values 1, 5, 3, 4,2 in this RDD
2  inputList = [1, 5, 3, 4, 2]
3  inputRDD = sc.parallelize(inputList)
4  # Retrieve the 2 smallest elements of the inputRDD and store them in
5  # a local python list
6  retrievedValues = inputRDD.takeOrdered(2)
7

```

Listing 26: Esempio takeOrdered

- **takeSample(withReplacement, num):** ritorna una lista python di oggetti contenenti num elementi random dell’RDD.

```

1  # Create an RDD of integers. Load the values 1, 5, 3, 3,2 in this RDD
2  inputList = [1, 5, 3, 3, 2]
3  inputRDD = sc.parallelize(inputList)
4  # Retrieve randomly two elements of the inputRDD and store them in
5  # a local python list
6  randomValues= inputRDD.takeSample(True, 2)
7

```

Listing 27: Esempio takeSample

- **reduce():** ritorna un singolo oggetto ottenuto combinando tutti gli oggetti di un RDD, ad esempio vogliamo la somma degli elementi, si passerà una funzione f come argomento di reduce. Nella funzione lambda avremo bisogno in questo caso di due argomenti che saranno quelli coinvolti nell’operazione. Presa una lista reduce prende due elementi alla volta (il primo e l’ultimo), ne sostituisce uno con il risultato parziale e rifà il procedimento fino ad ottenere un singolo elemento, che sarà quello ritornato.

```

1  # Create an RDD of integers. Load the values 1, 2, 3, 3 in this RDD
2  inputListReduce = [1, 2, 3, 3]
3  inputRDDReduce = sc.parallelize(inputListReduce)
4  # Compute the sum of the values
5  sumValues = inputRDDReduce.reduce(lambda e1, e2: e1+e2)
6

```

Listing 28: Esempio reduce

- **fold:** Se si ha una funzione che è associativa ma non è commutativa si può usare questa funzione. Ritorna un oggetto ottenuto dalla combinazione degli elementi dell’RDD. Come argomenti presenta un ZeroValue che è un valore di partenza usato per la funzione f passata come altro parametro. Ad esempio se vogliamo concatenare le stringhe di una lista, il valore di partenza sarà una stringa vuota a cui concateneremo le altre stringhe. Funzionalmente fold e reduce funzionano allo stesso modo, cambia il fatto che fold viene usato per funzioni che non sono commutative e per la presenza di un zero value.

```

1  # Create an RDD of integers. Load the values 1, 2, 3, 3 in this RDD
2  inputListFold = ['This ', 'is ', 'a ', 'test']
3  inputRDDFold = sc.parallelize(inputListFold)
4  # Concatenate the input strings
5  finalString = inputRDDFold.fold('', lambda s1, s2: s1+s2)
6

```

Listing 29: Esempio fold

- **aggregate**: L'operazione aggregate consente di combinare gli elementi di un RDD in un singolo oggetto Python, utilizzando un valore iniziale "zero" e due funzioni fornite dall'utente: **seqOp** una funzione per combinare un elemento di tipo T con uno di tipo U, creando un nuovo elemento di tipo U. Viene applicata per aggregare gli elementi dell'RDD all'accumulatore di ogni partizione. **combOp** è una funzione per combinare due elementi di tipo U, usata per aggregare i risultati parziali delle diverse partizioni. L'RDD viene suddiviso in più partizioni, ciascuna con una propria lista di elementi. All'interno di ogni partizione, viene calcolato un risultato parziale applicando ricorsivamente seqOp tra l'accumulatore locale e ogni elemento della partizione. Una volta ottenuti i risultati parziali per ogni partizione, combOp combina i risultati parziali di tipo U in un singolo risultato finale.

```

1      # Create an RDD of integers. Load the values 1, 2, 3, 3 in this RDD
2      inputListAggr = [1, 2, 3, 3]
3      inRDD = sc.parallelize(inputListAggr)
4
5      # Instantiate the zero value
6      # We use a tuple containing two values:
7      # (sum, number of represented elements)
8      zeroValue = (0, 0)
9
10     # Compute the sum of the elements in inputRDDAggr and count them
11     sumCount = inRDD.aggregate(zeroValue, \
12                               lambda acc, e: (acc[0]+e, acc[1]+1), \
13                               lambda p1, p2: (p1[0]+p2[0], p1[1]+p2[1]))
14

```

Listing 30: Esempio aggregate

7.4 RDD basati su coppie chiave-valore

Spark supporta l'esistenza di RDD basati su coppie di chiave valore. In python sono rappresentate da tuple. Gli RDD chiave valore sono caratterizzati da operazioni come reduceByKey, join ecc. L'idea di base è simile ai programmi basati su MapReduce di Hadoop. Il primo passo è quello di creare un RDD di coppie chiave valore. Da un altro RDD possiamo utilizzare metodi come map o flatMap per trasformare in un altro RDD. L'altro metodo è quello per cui possiamo ottenere un RDD da una lista locale coppie chiave valore usando il metodo parallelize(). Un esempio su come ottenere un RDD di coppie chiave valore è quello qui sotto illustrato in cui utilizziamo una funzione lambda che ritorna una tupla in cui in prima posizione abbiamo la chiave (un nome) e come seconda posizione il valore, in questo caso un intero.

```

1  # Read the content of the input textual file
2  namesRDD = sc.textFile("first_names.txt")
3  # Create an RDD of key-value pairs
4  nameOnePairRDD = namesRDD.map(lambda name: (name, 1))

```

Vediamo ora il caso di una flatMap, in cui analogamente alla map, si applica una funzione a ciascun elemento dell'RDD e restituisce un nuovo RDD in cui ogni elemento è il risultato della funzione applicata al singolo elemento dell'RDD di partenza ma a differenza del map questa funzione può restituire 0, uno o più valori e non per forza un valore. È più utile nelle operazioni di splitting delle stringhe e di gestione dei testi.

```

1  # Define the function associated with the flatMap transformation
2  def wordsOnes(line):

```

```

3     pairs = []
4     for word in line.split(' '):
5         pairs.append( (word, 1) )
6     return pairs
7 # Read the content of the input textual file
8 linesRDD = sc.textFile("document.txt")
9 # Create an RDD of key-value pairs based on the input document
10 # One pair (word,1) for each input word
11 wordOnePairRDD = linesRDD.flatMap(wordsOnes)

```

Possiamo anche creare la collezione locale dell’RDD basato su coppie chiave valore mediante il metodo `parallelize`:

```

1     # Create the local python list
2     nameAge = [ ("Paolo", 40), ("Giorgio", 22), ("Paolo", 35)]
3     # Create the RDD of pairs from the local collection
4     nameAgePairRDD = sc.parallelize(nameAge)

```

7.4.1 Trasformazioni di RDD di coppie chiave-valore

Vediamo ora delle funzioni di trasformazioni usate in questo ambito:

ReduceByKey Crea un nuovo RDD di coppie chiave valore per ogni chiave dell’RDD di input. Il valore è ottenuto dalla combinazione dei valori aventi la stessa chiave nell’RDD di partenza. Non cambia il tipo di dati.

```

1 # Create the local python list
2 nameAge = [ ("Paolo", 40), ("Giorgio", 22), ("Paolo", 35)]
3 # Create the RDD of pairs from the local collection
4 nameAgePairRDD = sc.parallelize(nameAge)
5 # Select for each name the lowest age value
6 youngestPairRDD= nameAgePairRDD.reduceByKey(lambda age1, age2:\ min(age1, age2))

```

foldByKey ha lo stesso obiettivo della `reduceByKey()`, ma è caratterizzato da uno zero value da cui partire e la funzione deve essere solo associativa e non per forza commutativa. Viene passata una funzione `op` che ha lo scopo di combinare i valori con lo zero value di partenza per ottenere il risultato aggregato.

```

1 # Create the local python list
2 nameMess = [ ("Paolo", "Message1"), ("Giorgio", "Message2"), ("Paolo", "Message3")]
3 # Create the RDD of pairs from the local collection
4 nameMessPairRDD = sc.parallelize(nameMess)
5 # Concatenate the messages of each user
6 concatPairRDD= nameMessPairRDD.foldByKey('', lambda m1, m2:m1+m2)

```

CombineByKey Crea un nuovo RDD di coppie chiave valore dopo c’è una coppia per ogni chiave distinta dell’RDD originale. Il valore associato potrà avere un tipo diverso rispetto a quello di prima. La `createCombiner` passata tra i parametri è la funzione che prende i valori di partenza e trasforma in valori di tipo diverso. La `mergeValue` è la funzione che contiene il codice per combinare un valore del nuovo tipo con un valore del tipo originario. La `mergeCombiner` è la funzione che mette insieme due valori del nuovo tipo.

```

1 # Create the local python list
2 nameAge = [ ("Paolo", 40), ("Giorgio", 22), ("Paolo", 35)]
3 # Create the RDD of pairs from the local collection
4 nameAgePairRDD = sc.parallelize(nameAge)
5 # Compute the sum of ages and
6 # the number of input pairs for each name (key)
7 sumNumPerNamePairRDD=nameAgePairRDD.combineByKey( \
8     lambda inputElem: (inputElem, 1), \
9     lambda intermediateElem, inputElem: \
10 (intermediateElem[0]+inputElem, intermediateElem[1]+1),
11     lambda intermediateElem1, intermediateElem2: \ (intermediateElem1[0]+
12         intermediateElem2[0], \ intermediateElem1[1]+intermediateElem2[1]) )

```

GroupByKey Crea un gruppo per ogni chiave distinta e il valore è un'iterable di tutti i valori con la stessa chiave. Non presenta parametri in quanto l'obiettivo è sempre questo.

```

1 # Create the local python list
2 nameAge = [ ("Paolo", 40), ("Giorgio", 22), ("Paolo", 35)]
3 # Create the RDD of pairs from the local collection
4 nameAgePairRDD = sc.parallelize(nameAge)
5 # Create one group for each name with the list of associated ages
6 agesPerNamePairRDD = nameAgePairRDD.groupByKey()
7 # Store the result in an output folder
8 agesPerNamePairRDD.mapValues(lambda listValues: list(listValues)).saveAsTextFile(
9     outputPath);

```

Mapvalues Ritorna un nuovo RDD basato su coppie applicando una funzione ad ogni coppia dell'RDD di partenza andando a modificare i valori indifferentemente dalla chiave. Quindi se noi vogliamo incrementare di uno tutti i valori possiamo farlo con questa funzione.

```

1 # Create the local python list
2 nameAge = [ ("Paolo", 40), ("Giorgio", 22), ("Paolo", 35)]
3 # Create the RDD of pairs from the local collection
4 nameAgePairRDD = sc.parallelize(nameAge)
5 # Increment age of all users
6 plusOnePairRDD = nameAgePairRDD.mapValues(lambda age: age+1)
7 # Save the result on disk
8 plusOnePairRDD.saveAsTextFile(outputPath)
9

```

Keys Ritorna un RDD non più di coppie chiave valori ma solamente di chiavi. I duplicati non vengono eliminati.

```

1 # Create the local python list
2 nameAge = [ ("Paolo", 40), ("Giorgio", 22), ("Paolo", 35)]
3 # Create the RDD of pairs from the local collection
4 nameAgePairRDD = sc.parallelize(nameAge)
5 # Select the key part of the input RDD of key-value pairs
6 namesRDD = nameAgePairRDD.keys().distinct()
7 # Store the result in an output folder
8 namesRDD.saveAsTextFile(outputPath);
9

```

Values Esiste il metodo duale che ritorna un RDD di valori da un RDD di coppie chiave valore. I duplicati anche qui non sono eliminati.

```

1 # Create the local python list
2 nameAge = [ ("Paolo", 40), ("Giorgio", 22), ("Paolo", 22)]
3 # Create the RDD of pairs from the local collection
4 nameAgePairRDD = sc.parallelize(nameAge)
5 # Select the value part of the input RDD of key-value pairs
6 agesRDD = nameAgePairRDD.values()
7 # Store the result in an output folder
8 agesRDD.saveAsTextFile(outputPath);
9

```

SortByKey le coppie vengono ordinate in base alla chiave in maniera ascendente.

```

1 # Create the local python list
2 nameAge = [ ("Paolo", 40), ("Giorgio", 22), ("Paolo", 35)]
3 # Create the RDD of pairs from the local collection
4 nameAgePairRDD = sc.parallelize(nameAge)
5 # Sort by name the content of the input RDD of key-value pairs
6 sortedNameAgePairRDD = nameAgePairRDD.sortByKey()
7 # Store the result in an output folder
8 sortedNameAgePairRDD.saveAsTextFile(outputPath);
9

```

7.4.2 Trasformazioni tra due RDD

Analogamente a quanto visto prima, possiamo usare delle funzioni tra RDD basati su coppie chiave valore per ottenere dati aggregati.

SubtractByKey Si ha un nuovo RDD che contiene solo le coppie di chiave valore del primo RDD che non sono presenti nel secondo RDD.

```

1 profiles = [ ("PaoloG", 40), ("Giorgio", 22), ("PaoloB", 35)]
2 # Create the RDD of pairs from the profiles local list
3 profilesPairRDD = sc.parallelize (profiles)
4 # Create the second local python list
5 banned = [ ("PaoloB", "spam"), ("Giorgio", "Vandalism")]
6 # Create the RDD of pairs from the banned local list
7 bannedPairRDD = sc.parallelize (banned)
8 # Select the profiles of the ?good? users
9 selectedProfiles = profilesPairRDD.subtractByKey(bannedPairRDD)
10

```

(Inner) Join È usato per le operazioni di Join tra le coppie degli RDD. Il risultato delle coppie saranno coppie con la stessa chiave e come valore la tupla dei due valori.

```

1 # Create the first local Python list
2 questions= [(1, "What is .. ?"), (2, "Who is ..?")]
3 # Create the RDD of pairs from the local list
4 questionsPairRDD = sc.parallelize (questions)
5 # Create the second local python list
6 answers = [(1, "It is a car"), (1, "It is a byke"), (2, "She is Jenny")]
7 # Create the RDD of pairs from the local list

```

```

8  answersPairRDD = sc.parallelize(answers)
9  # Join questions with answers
10 joinPairRDD = questionsPairRDD.join(answersPairRDD)
11

```

CoGroup Raggruppa nella stessa chiave, la lista con la corrente chiave degli elementi aventi quella chiave del primo RDD e del secondo RDD in una tupla. Quindi come valore avremo una tupla di due liste.

```

1  # Create the first local python list
2  movies= [(1, "Star Trek"), (1, "Forrest Gump"), (2, "Forrest Gump")]
3  # Create the RDD of pairs from the first local list
4  moviesPairRDD = sc.parallelize(movies)
5  # Create the second local python list
6  directors = [ (1, "Woody Allen"), (2, "Quentin Tarantino"), (2, "Alfred Hitchcock")]
7  # Create the RDD of pairs from the second local list
8  directorsPairRDD = sc.parallelize(directors)
9  # Cogroup movies and directors per user
10 cogroupPairRDD = moviesPairRDD.cogroup(directorsPairRDD)

```

7.4.3 Azioni di RDD di coppie chiave-valore

Oltre alle trasformazioni abbiamo anche le azioni su RDD di coppie chiave valore:

countByKey : abbiamo un input di RDD di coppie chiave valore, quello che viene ritornato è un dizionario contenente le informazioni sul numero di elementi associato a quella chiave. Bisogna fare attenzione alla dimensione del risultato perché se troppo grande, l'azione non può essere memorizzata come variabile locale del driver.

```

1  # Create the local python list
2  movieRating= [("Forrest Gump", 4), ("Star Trek", 5), ("Forrest Gump", 3)]
3  # Create the RDD of pairs from the local collection
4  movieRatingRDD = sc.parallelize(movieRating)
5  # Compute the number of rating for each movie
6  movieNumRatings = movieRatingRDD.countByKey()
7  # Print the result on the standard output
8  print(movieNumRatings)
9

```

collectAsMap : abbiamo un input di RDD di coppie chiave valore, ritorna un dizionario in cui si ha una copia di ogni coppia contenuta nell'RDD di partenza. Anche in questo caso bisogna fare attenzione alla dimensione dell'RDD. La differenza con la collect sta nel fatto che collect ritorna una lista di tuple mentre collectAsMap ritorna un dizionario.

```

1  # Create the local python list
2  users = [("User1", "Paolo"), ("User2", "Luca"), ("User3", "Daniele")]
3  #Create the RDD of pairs from the local list
4  usersRDD = sc.parallelize(users)
5  # Retrieve the content of usersRDD and store it in a
6  # local python dictionary
7  retrievedPairs = usersRDD.collectAsMap()
8  # Print the result on the standard output
9  print(retrievedPairs)
10

```

lookup : ritorna una lista locale contenente i valori associati alla chiave messa nei parametri.

```
1  # Create the local python list
2  movieRating=  [("Forrest Gump", 4), ("Star Trek", 5), ("Forrest Gump", 3)]
3  # Create the RDD of pairs from the local collection
4  movieRatingRDD = sc.parallelize(movieRating)
5  # Select the ratings associated with ?Forrest Gump?
6  movieRatings = movieRatingRDD.lookup("Forrest Gump")
7  # Print the result on the standard output
8  print(movieRatings)
9
```

8 Persistenza e cache

Spark utilizza un approccio chiamato calcolo lazy per la gestione degli RDD (Resilient Distributed Dataset). Questo significa che il contenuto di un RDD non viene calcolato immediatamente quando viene creato, ma solo nel momento in cui viene invocata un'azione, come collect, count o save. Le azioni forzano Spark a eseguire le trasformazioni precedenti per calcolare effettivamente i dati richiesti. Tuttavia, ogni volta che un'azione viene eseguita su un RDD o su un suo "discendente" (cioè un RDD derivato tramite trasformazioni come map o filter), Spark ricalcola completamente il contenuto dell'RDD dalla sua origine. Questo comportamento, pur essendo fondamentale per l'efficienza in alcuni scenari, diventa costoso in applicazioni iterative, dove gli stessi RDD vengono utilizzati ripetutamente.

Per affrontare questo problema, Spark offre la possibilità di persistere o memorizzare nella cache gli RDD. Persistendo o cacheando un RDD, ogni nodo del cluster mantiene in memoria il contenuto delle sue partizioni una volta calcolato per la prima volta in seguito a un'azione. Successivamente, qualsiasi altra azione che coinvolga lo stesso RDD o uno derivato utilizzerà il contenuto già memorizzato, anziché ricalcolarlo. Questo meccanismo permette di evitare il ri-calcolo ripetitivo delle trasformazioni, migliorando significativamente le prestazioni, soprattutto per applicazioni iterative, dove il guadagno può superare spesso un fattore di 10 in termini di velocità.

Quando si decide di persistere o cacheare un RDD, Spark utilizza i nodi del cluster per immagazzinare le partizioni. Di default, i dati vengono memorizzati nella memoria principale dei nodi. Tuttavia, Spark supporta diversi livelli di storage che permettono di specificare come e dove i dati persistono. Ad esempio, i dati possono essere mantenuti interamente in memoria, parzialmente in memoria e su disco (quando la memoria non è sufficiente), oppure interamente su disco. Questa flessibilità consente agli sviluppatori di adattare l'utilizzo delle risorse del cluster alle necessità specifiche delle applicazioni, bilanciando l'uso di memoria e disco.

In sintesi, la cache e la persistenza degli RDD sono strumenti cruciali per migliorare l'efficienza di Spark, consentendo di ottimizzare il calcolo dei dati attraverso un approccio che evita ricalcoli inutili, riduce i tempi di esecuzione e sfrutta al meglio le risorse hardware disponibili.

In Spark, il concetto di storage level è fondamentale per ottimizzare l'uso delle risorse durante il calcolo distribuito, specialmente in scenari in cui un RDD viene utilizzato più volte. Di default, Spark calcola il contenuto di un RDD ogni volta che viene invocata un'azione. Questo approccio può risultare inefficiente in applicazioni iterative o in cui lo stesso RDD è usato ripetutamente. Per migliorare le prestazioni, Spark offre due meccanismi principali: persistenza e cache. Entrambi permettono di memorizzare i dati intermedi calcolati per riutilizzarli in operazioni successive.

Livelli di storage di Spark I livelli di storage definiscono dove e come i dati dell'RDD vengono memorizzati. Spark supporta diverse modalità di persistenza, ciascuna con caratteristiche specifiche:

- **MEMORY_ONLY**: memorizza le partizioni dell'RDD come oggetti Java deserializzati nella memoria della JVM. Se la memoria non è sufficiente, alcune partizioni non vengono cacheate e vengono ricalcolate ogni volta che sono necessarie. Questo è il livello predefinito.

- **MEMORY_AND_DISK**: memorizza le partizioni nella memoria della JVM. Se non c'è spazio sufficiente in memoria, le partizioni in eccesso vengono salvate sul disco locale del nodo. In caso di necessità, vengono lette dal disco.
- **DISK_ONLY**: memorizza tutte le partizioni dell'RDD solo su disco locale. È utile quando la memoria è un fattore limitante.
- **MEMORY_ONLY_2** e **MEMORY_AND_DISK_2**: simili ai livelli sopra, ma replicano ciascuna partizione su due nodi del cluster. Questo aumenta la tolleranza ai guasti, poiché un nodo alternativo può essere utilizzato per accedere ai dati se uno dei due nodi fallisce.
- **OFF_HEAP** (sperimentale): memorizza i dati nella memoria off-heap, anziché nella memoria heap della JVM. Richiede l'abilitazione della memoria off-heap ed è utile per ridurre la pressione del garbage collector.

La scelta del livello di storage dipende dai requisiti dell'applicazione, come la disponibilità di memoria e la frequenza con cui un RDD deve essere ricalcolato.

8.1 Utilizzo di persist e cache

Per memorizzare un RDD, si può utilizzare il metodo `persist(storageLevel)` specificando il livello di storage desiderato. Un livello comune è `pyspark.StorageLevel.MEMORY_AND_DISK`, che garantisce un bilanciamento tra prestazioni e affidabilità. Il metodo `cache()`, invece, è una shortcut per persistere un RDD con il livello `MEMORY_ONLY`. Quando un RDD viene persiste o cacheato:

- Alla prima azione, il contenuto dell'RDD viene calcolato e memorizzato.
- Le azioni successive utilizzano il contenuto memorizzato, evitando calcoli ripetuti e migliorando significativamente le prestazioni.

Poiché gli RDD sono immutabili, sia `persist` che `cache` restituiscono un nuovo RDD, lasciando invariato l'originale.

8.2 Rimozione manuale e automatica dei dati

Spark utilizza una strategia LRU (Least Recently Used) per monitorare la cache su ciascun nodo. Le partizioni meno recenti vengono rimosse dalla cache quando lo spazio è insufficiente. È possibile rimuovere manualmente un RDD dalla cache con il metodo `unpersist()`. Efficienza del caching su disco Persistere gli RDD su disco è vantaggioso solo se:

- L'RDD è significativamente più piccolo rispetto al dataset originale.
- Le funzioni utilizzate per calcolare l'RDD sono costose. In caso contrario, ricalcolare i dati potrebbe essere veloce quanto leggerli dal disco.

```
1 # Legge il contenuto di un file di testo e lo cachea
2 inputRDD = sc.textFile("words.txt").cache()
3
4 # Conta il numero di righe e il numero di parole distinte
5 print("Numero di parole:", inputRDD.count())
6 print("Numero di parole distinte:", inputRDD.distinct().count())
```

Listing 31: Esempio pratico: caching di un RDD

In questo esempio, la prima azione (`count()`) calcola il contenuto dell’RDD leggendo il file e memorizza i dati in memoria. La seconda azione (`distinct().count()`) esegue le operazioni successivamente sulla memoria, anziché rileggere il file di input, migliorando le prestazioni. Vantaggi della cache/persistenza Il caching/persistenza è particolarmente utile quando:

- Lo stesso RDD viene utilizzato in più azioni o trasformazioni.
- Le operazioni su un RDD sono costose.
- Si vogliono evitare calcoli ridondanti in applicazioni iterative.

Utilizzando correttamente i livelli di storage e le strategie di cache, Spark può gestire in modo efficiente grandi volumi di dati, ottimizzando il tempo di esecuzione e l’utilizzo delle risorse del cluster.

8.3 Accumulatori

Qualche volta vogliamo analizzare dei dati per ottenere delle statistiche. Gli accumulatori sono usati per creare delle variabili condivise pensate per operazioni associative che possono essere eseguite in parallelo. Possono essere usate per implementare contatori o somme. Il valore dell’accumulatore è incrementato nella funzione associata con la trasformazione. Abbiamo degli accumulatori numerici di integers o floats. Per definire un accumulatore si usa il metodo `accumulator(value)` e il sistema definisce un oggetto accumulatore. L’oggetto è memorizzato nella memoria principale. Quando vogliamo incrementare il valore dell’accumulatore viene usato il metodo `add`. Supponiamo di avere una lista di email. Vogliamo ora contare il numero di email non valide (non contenenti @). Quello che facciamo nel codice è inizializzare un accumulatore e richiamarlo attraverso la funzione `add` all’interno della funzione usata poi per la filter. Alla fine del salvataggio dell’RDD in un file, stampiamo il valore dell’accumulatore. Possiamo creare degli accumulatori personalizzati di un tipo T ma non è una soluzione consigliata.

```

1      # Define an accumulator. Initialize it to 0
2      invalidEmails = sc.accumulator(0)
3
4      # Read the content of the input textual file
5      emailsRDD = sc.textFile("emails.txt")
6
7      #Define the filtering function
8      def validEmailFunc(line):
9          if (line.find('@')<0):
10             invalidEmails.add(1)
11         return False
12         else:
13             return True
14
15     # Select only valid emails
16     # Count also the number of invalid emails
17     validEmailsRDD = emailsRDD.filter(validEmailFunc)
18

```

Listing 32: Esempio accumulatori

9 Spark SQL

9.1 DataFrames

Spark SQL è una parte di Spark che permette di usare l'SQL. Il sistema per rendere tutto più efficiente utilizza un ottimizzatore. Vengono utilizzati dei moduli chiamati DataFrames che sono delle collezioni di dati strutturati. Sono concettualmente equivalenti alle tabelle relazionali infatti possono essere visti come una lista di righe. Possono essere creati da CSV, Json. All'inizio viene

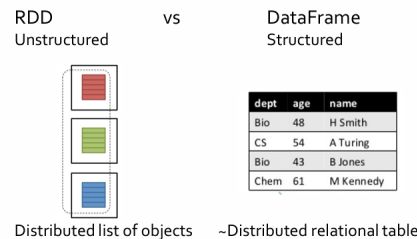


Figura 10: RDD vs Dataframe

creato un oggetto SparkSession per accedere alle funzionalità di Spark SQL. Per chiudere la sessione SparkSession si usa `.stop()`.

```
1 #importiamo la SparkSession
2 from pyspark.sql import SparkSession
3 #Creiamo l'istanza
4 spark = SparkSession.builder.getOrCreate()
```

9.2 Creare un DataFrames

Possiamo creare un DataFrames partendo da un file strutturati di testo come csv, json ma anche da RDD già esistenti. Per creare un DataFrames da un CSV si può usare:

```
1 spark.read.load(path, options)
```

Dove path è il percorso del file di input e options è legato alla presenza o meno di un header e spark è un oggetto spark session. Per un esempio più completo abbiamo:

```
1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load("persons.csv",
5                       format="csv",
6                       header=True,
7                       inferSchema=True)
8
```

Come vediamo oltre alla gestione dell'header nelle options abbiamo anche il formato. `inferSchema` si usa per specificare che il sistema deve dedurre il tipo di dati per ogni colonna. Senza questa opzione i dati vengono visti tutti come stringhe. Se vogliamo creare un DataFrames da un JSON avremo la scrittura analoga ma con la differenza nel format:

```

1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load( "persons.json",
5                       format="json")
6

```

Un'altra variante è quello di impostare al posto del nome del file, il nome di una cartella che contiene un set di files JSON.

```

1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load( "folder_JSONFiles/",
5                       format="json",
6                       multiLine=True)
7

```

Data una lista locale in python, possiamo impostare un DataFrames utilizzando il metodo createDataFrames che prende come parametri, il nome della lista e uno schema su come impostare i dati all'interno del DataFrame.

```

1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a Python list of tuples
4 profilesList = [(19, "Justin"), (30, "Andy"), (None, "Michael")]
5 # Create a DataFrame from the profilesList
6 df = spark.createDataFrame(profilesList, ["age", "name"])
7

```

Questo può essere particolarmente utile anche per memorizzare in un DataFrame il contenuto di un RDD. Possiamo fare anche l'operazione opposta ovvero portare il contenuto di un DataFrame in un RDD basato su oggetti Row. Un oggetto Row è come un dizionario contenente il nome delle colonne in questione come chiavi e i valori delle colonne come valori dell'appunto dizionario. I campi della classe Row possono essere acceduti o come attributi con la notazione row.key oppure come un dizionario row["key"]. Il metodo addDict() ritorna il contenuto di Row come dizionario.

Vediamo ora un esempio:

```

1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load( "persons.csv",
5                       format="csv",
6                       header=True,
7                       inferSchema=True)
8 # Define an RDD based on the content of the DataFrame
9 rddRows = df.rdd
10 # Use the map transformation to extract
11 # the name field/column
12 rddNames = rddRows.map(lambda row: row.name)
13 # Store the result
14 rddNames.saveAsTextFile(outputPath)
15

```

9.3 Operazioni sui Dataframes

Sappiamo creare un dataframe, vogliamo ora gestirlo per ricavare risultati e ottenere ad esempio statistiche. I metodi che possiamo usare sono:

Show stampa nello standard output i primi n record del dataframe.

```
1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load( "persons.csv",
5 format="csv",
6 header=True,
7 inferSchema=True)
8 df.show(2)
```

printSchema stampa nello standard output lo schema del dataframe.

count ritorna il numero di record

distinct ritorna un nuovo dataframe privo di duplicati

```
1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from names.csv
4 df = spark.read.load( "names.csv",
5 format="csv",
6 header=True,
7 inferSchema=True)
8 df_distinct = df.distinct()
```

Select(col1,...,coln) ritorna un nuovo dataframe contenente solo le colonne specificate dal dataframe di input

```
1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons2.csv
4 df = spark.read.load( "persons2.csv",
5 format="csv",
6 header=True,
7 inferSchema=True)
8 dfNamesAges = df.select("name", "age")
```

SelectExpr(expression1,...,expressionN) è una variante della select ma con l'aggiunta di espressioni che possono contenere query SQL.

```
1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load( "persons2.csv",
5 format="csv",
6 header=True,
7 inferSchema=True)
```

```

8     # Create a new DataFrame with four columns:
9     # name, age, gender, newAge = age + 1
10    dfNewAge = df.selectExpr("name", "age", "gender",
11                             "age+1 as newAge")

```

filter ritorna una dataframe che contiene solo le righe che soddisfano la condizione.

```

1     # Create a Spark Session object
2     spark = SparkSession.builder.getOrCreate()
3     # Create a DataFrame from persons.csv
4     df = spark.read.load("persons.csv",
5                          format="csv",
6                          header=True,
7                          inferSchema=True)
8     df_filtered = df.filter("age>=20 and age<=31")

```

Join(right, on,how) è un metodo che è usato per fare una join tra due dataframes. on è la condizione della join e how è il tipo di join che può essere inner, cross, full, full_outer, left, left_outer ecc.

```

1     # Create a Spark Session object
2     spark = SparkSession.builder.getOrCreate()
3
4     # Read persons_id.csv and store it in a DataFrame
5     dfPersons = spark.read.load("persons_id.csv",
6                                 format="csv",
7                                 header=True,
8                                 inferSchema=True)
9
10    # Read liked_sports.csv and store it in a DataFrame
11    dfUidSports = spark.read.load("liked_sports.csv",
12                                  format="csv",
13                                  header=True,
14                                  inferSchema=True)
15
16    # Join the two input DataFrames
17    dfPersonLikes = dfPersons.join(dfUidSports, dfPersons.uid == dfUidSports.uid)
18
19    # Print the result on the standard output
20    dfPersonLikes.show()

```

9.4 Funzioni di aggregazioni

Ci sono funzioni per calcolare dati aggregati da applicare su set di valori dei dataframes. Utilizziamo la funzione **agg** per utilizzare una delle funzioni aggregate qui sotto. L'argomento di agg sarà quindi: le colonne che prendiamo e il tipo di funzione aggregata. Ad esempio se vogliamo calcolare la media delle età abbiamo la scrittura:

```

1 averageAge = df.agg({"age": "avg"})

```

Quello che possiamo fare con agg è avere più parametri all'interno della funzione, in particolare due funzioni aggregate

```

1 dfAverageMaxAge=df.agg(F.avg("age"),F.max("age"))
2

```

Il risultato sarò un dataframe con due colonne, una per avg e una per max. Le altre funzioni aggregate sono:

- `avg(nome_colonna)`: ritorna un dataset con la media dei valori

```
1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load( "persons.csv",
5 format="csv",
6 header=True,
7 inferSchema=True)
8 # Compute the average of age
9 averageAge = df.agg({"age": "avg"})
10
```

- `groupBy(col1,...,coln)`: l'output è un dataframe con i record raggruppati in base agli attributi.

```
1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load( "persons.csv",
5 format="csv",
6 header=True,
7 inferSchema=True)
8 grouped = df.groupBy("name").avg("age")
9
```

- `sort(col1,...,coln)`: è un metodo che ritorna un nuovo dataframe con i record ordinati in base ai parametri della funzione.

9.5 Dataframe e SQL

Spark permette di utilizzare query sul contenuto di un Dataframe utilizzando linguaggio SQL. Vediamo ora alcuni metodi:

`createOrReplaceTempView` crea una tabella temporanea.

`sql(query)` è un metodo di `sparkSession` in cui mettiamo tra i parametri una funzione `sql`.

```
1 # Create a Spark Session object
2 spark = SparkSession.builder.getOrCreate()
3 # Create a DataFrame from persons.csv
4 df = spark.read.load(
5 "persons.json",
6 format="json")
7 # Assign the ?table name? people to the df DataFrame
8 df.createOrReplaceTempView("people");
9 # Select the persons with age between 20 and 31
10 # by querying the people table
11 selectedPersons =
12 spark.sql("SELECT * FROM people WHERE age>=20 and
13 age<=31")
14 # Print the result on the standard output
15 selectedPersons.show()
```

16

Listing 33: utilizzo di createOrReplaceTempView e sql

9.6 Approcci alla Memorizzazione

I contenuti dei DataFrame possono essere memorizzati su disco utilizzando due metodi principali:

1. Conversione in RDD e utilizzo di `saveAsTextFile`:

- Un DataFrame può essere convertito in un RDD tradizionale utilizzando il metodo `.rdd`.
- Successivamente, si può utilizzare il metodo `saveAsTextFile(outputFolder)` per salvare i dati in una cartella di output.

2. Utilizzo del metodo `write`:

- Il metodo `.write()` dei DataFrame restituisce un'istanza della classe `DataFrameWriter`.
- Questo metodo consente di salvare i dati in diversi formati, come CSV, JSON, Parquet, ecc.

```
1 from pyspark.sql import SparkSession
2
3 spark = SparkSession.builder.getOrCreate()
4 df = spark.read.load("persons.csv", format="csv", header=True, inferSchema=True)
5 df.rdd.saveAsTextFile(outputPath)
```

Un altro metodo di salvataggio è `write`:

- Come nel caso precedente, si parte dal file `persons.csv`.
- Questa volta, il salvataggio avviene utilizzando il metodo `write`, specificando il formato CSV:

```
1 df.write.csv(outputPath, header=True)
```

9.7 Funzioni Predefinite e Funzioni Definite dall'Utente (UDF)

Spark SQL fornisce diverse funzioni di sistema, come:

- `hour(Timestamp)`: estrae l'ora da un timestamp.
- `abs(Integer)`: calcola il valore assoluto.

Queste funzioni possono essere utilizzate in trasformazioni come `selectExpr()` o in query SQL. Gli utenti possono definire UDF personalizzate tramite il metodo `.udf.register()`.

Parametri principali:

- `name`: nome della funzione UDF.
- `function`: funzione che definisce il comportamento.
- `datatype`: tipo di dato SQL del valore restituito.

Una UDF prende uno o più parametri come input e restituisce un singolo valore.

Facciamo ora un esempio in cui vogliamo definire una UDF che calcola la lunghezza di una stringa:

```
1 spark.udf.register("length", lambda x: len(x))
```

Listing 34: Esempio con len

```
1 result = inputDF.selectExpr("length(name) as size")
```

Listing 35: Esempio con selectExpr

```
1 result = spark.sql("SELECT length(name) FROM profiles")
```

Listing 36: Esempio con query sql

9.8 Operazioni di Set sui DataFrame

I DataFrame supportano operazioni di insieme simili a quelle degli RDD:

- **union**: unisce i dati di due DataFrame.
- **intersect**: restituisce gli elementi comuni.
- **subtract**: sottrae i dati di un DataFrame dall'altro.

9.9 Join e Broadcast

Spark SQL può implementare automaticamente un broadcast join se uno dei DataFrame è abbastanza piccolo da essere memorizzato nella memoria principale di ogni executor. È possibile forzare l'operazione di broadcast utilizzando la funzione **broadcast**:

```
1 from pyspark.sql.functions import broadcast
2
3 dfPersonLikesBroadcast = dfUidSports.join(
4     broadcast(dfPersons),
5     dfPersons.uid == dfUidSports.uid
6 )
```

L'operazione di broadcast ottimizza i join distribuendo i dati del DataFrame più piccolo su tutti gli executor.

9.10 Piano di Esecuzione

Il metodo **explain()** può essere invocato su un DataFrame per stampare il piano di esecuzione del calcolo:

```
1 df.explain()
```

Listing 37: Esempio explain

Questo è utile per analizzare e ottimizzare le prestazioni delle query.

9.11 Conclusioni e Recensione Finale

Sei arrivato alla fine del programma, la parte dopo non è presente all'esame ma è ugualmente consultabile in caso di curiosità. Spero che questi appunti ti siano serviti, in ogni caso se vuoi lasciare una recensione, ti lascio il link del canale telegram **CLICCA QUI**, ti auguro un buon proseguimento degli studi

10 Apache Spark MLlib

Apache Spark MLlib è una libreria integrata in Spark, sviluppata per fornire strumenti avanzati di machine learning e data mining, consentendo l'elaborazione di grandi dataset distribuiti. Il suo obiettivo principale è semplificare l'uso di algoritmi complessi su infrastrutture distribuite, rendendoli accessibili attraverso un'architettura altamente scalabile. MLlib supporta due principali API: `pyspark.mllib`, basata sugli RDDs (Resilient Distributed Datasets), e `pyspark.ml`, che sfrutta i DataFrame come struttura dati principale. Quest'ultima è preferita per la sua maggiore flessibilità e per l'adozione del concetto di pipeline, che semplifica la definizione e l'esecuzione di flussi di lavoro complessi.

10.1 Rappresentazione dei dati

Un elemento cruciale di MLlib è la rappresentazione dei dati, che include tipi locali (vettori e matrici) e distribuiti. I vettori locali sono utilizzati per rappresentare le caratteristiche numeriche di un record. Possono essere:

- **Densi:** in cui tutti i valori sono rappresentati esplicitamente, ad esempio, `[1.0, 0.0, 3.0]`.
- **Sparsi:** in cui solo gli indici e i valori non nulli sono memorizzati per ottimizzare l'uso dello spazio, ad esempio, `(3, [0, 2], [1.0, 3.0])`, dove:
 - 3 è la dimensione del vettore.
 - `[0, 2]` sono gli indici dei valori non nulli.
 - `[1.0, 3.0]` sono i valori associati a quegli indici.

Esempio di codice per creare vettori:

```
1 from pyspark.ml.linalg import Vectors
2
3 # Creazione di un vettore denso
4 denso = Vectors.dense([1.0, 0.0, 3.0])
5
6 # Creazione di un vettore sparso
7 sparso = Vectors.sparse(3, {0: 1.0, 2: 3.0})
8
9 print(f"Vettore denso: {denso}")
10 print(f"Vettore sparso: {sparso}")
```

10.2 Trasformazione dei dati grezzi

Spark MLlib richiede che i dati grezzi vengano trasformati in strutture tabellari (`DataFrame`), dove ogni riga rappresenta un record e ogni colonna ha un significato specifico. Ad esempio:

- La colonna `features` contiene un vettore con i valori delle caratteristiche numeriche.
- La colonna `label` rappresenta il target in problemi di classificazione o regressione.

La preparazione dei dati include operazioni di normalizzazione, trasformazione e selezione delle caratteristiche, ognuna delle quali può essere implementata attraverso specifici Transformer o Estimator.

I **Transformer** sono oggetti che operano su un DataFrame, trasformandolo in un nuovo DataFrame con nuove colonne aggiuntive. Un esempio pratico è l'uso di VectorAssembler, che combina più colonne di input in un'unica colonna vettoriale, utile per creare rappresentazioni compatte delle caratteristiche. Gli **Estimator**, invece, rappresentano l'astrazione gli algoritmi di apprendimento, come la regressione logistica o il clustering, che generano modelli applicabili ai dati. Il processo di apprendimento prevede che un Estimator venga allenato sui dati di training, producendo un modello trasformabile. Questo modello viene allenato con i dati su cui vogliamo basarci attraverso il metodo fit che produce un Transformer che verrà poi utilizzato per creare il nuovo Dataframe.

Il concetto di pipeline è particolarmente potente, poichè consente di concatenare più operazioni, sia di trasformazione che di apprendimento, in un'unica struttura sequenziale.

La preparazione dei dati comprende operazioni come:

- **Normalizzazione**, per uniformare le scale dei dati.
- **Estrazione di caratteristiche**, come TF-IDF per dati testuali.
- **Selezione di caratteristiche**, per ridurre il numero di dimensioni e migliorare l'efficienza.

Esempio di normalizzazione con StandardScaler:

```
1 from pyspark.ml.feature import StandardScaler
2
3 # DataFrame di esempio
4 data = [(0, Vectors.dense([1.0, 0.5, -1.0])),
5         (1, Vectors.dense([2.0, 1.0, 1.0]))]
6 df = spark.createDataFrame(data, ["id", "features"])
7
8 # Creazione dello scaler
9 scaler = StandardScaler(inputCol="features", outputCol="scaledFeatures", withStd=
10                          True, withMean=False)
11
12 # Addestramento dello scaler
13 scalerModel = scaler.fit(df)
14
15 # Trasformazione dei dati
16 scaledData = scalerModel.transform(df)
17 scaledData.show()
```

10.3 Pipeline di machine learning

Il concetto di **pipeline** consente di concatenare più fasi in un'unica struttura, integrando la preparazione dei dati, l'addestramento del modello e la predizione. Ad esempio, in un processo di classificazione testuale, si possono:

1. Tokenizzare i testi.
2. Convertire i token in vettori numerici.
3. Addestrare un modello di classificazione.

Esempio di pipeline:

```

1 from pyspark.ml import Pipeline
2 from pyspark.ml.classification import LogisticRegression
3 from pyspark.ml.feature import Tokenizer, HashingTF
4
5 # Dati di esempio
6 data = [(0, "Spark is great", 1.0), (1, "I love Spark", 0.0)]
7 df = spark.createDataFrame(data, ["id", "text", "label"])
8
9 # Componenti della pipeline
10 tokenizer = Tokenizer(inputCol="text", outputCol="words")
11 hashingTF = HashingTF(inputCol="words", outputCol="features")
12 lr = LogisticRegression(featuresCol="features", labelCol="label")
13
14 # Creazione della pipeline
15 pipeline = Pipeline(stages=[tokenizer, hashingTF, lr])
16
17 # Addestramento del modello
18 model = pipeline.fit(df)
19
20 # Applicazione su nuovi dati
21 predictions = model.transform(df)
22 predictions.show()

```

10.4 Feature Transformations

Tra le tecniche di pre-processing abbiamo le feature transformations per creare nuove colonne combinando o lavorando sulle colonne (features) già esistenti. Possiamo creare features o trasformarle usando i metodi che già conosciamo sui dataframe e RDD.

10.4.1 Vector Assembler

Vector Assembler è un transformer che combina una lista di colonne in una singola colonna. È utile per combinare in una singola colonna prima di applicare algoritmi di machine learning.

Prendiamo ora il codice seguente:

```

1 from pyspark.mllib.linalg import Vectors
2 from pyspark.ml.feature import VectorAssembler
3 # input and output folders
4 inputPath = "data/exampleDataAssembler.csv"
5 # Create a DataFrame from the input data
6 inputDF = spark.read.load(inputPath,\
7     format="csv", header=True, inferSchema=True)
8 # Create a VectorAssembler that combines columns colB and colC
9 # The new vector column is called features
10 myVectorAssembler = VectorAssembler(inputCols = ['colB', 'colC'],\
11     outputCol = 'features')
12 # Apply myVectorAssembler on the input DataFrame
13 transformedDF = myVectorAssembler.transform(inputDF)

```

La sintassi del Vector Assembler è **VectorAssembler(inputCols, outputCol)** dove inputCols è la lista delle colonne di partenza da includere nella creazione della nuova colonna di tipo Vector. Il tipo di colonne messe come input sono colonne di tipo numerico, booleano, vector.

outputCol è il nome della nuova colonna. Quando viene chiamato il metodo transform del Vector Assembler creato prima, che ha come parametro il dataframe di partenza, l'output sarà un nuovo Dataframe con la nuova feature.



colA	colB	colC
1	4.5	True
2	0.6	True
3	1.5	False
4	12.1	True
5	0.0	True

colA	colB	colC	features
1	4.5	True	[4.5,1.0]
2	0.6	True	[0.6,1.0]
3	1.5	False	[1.5,0.0]
4	12.1	True	[12.1,1.0]
5	0.0	True	[0.0,1.0]

Figura 11: Vector Transformation

Come vediamo nell'esempio, il contenuto della feature può essere sotto formato di vettore.

10.5 Normalizzazione

MLlib possiede un set di algoritmi per la normalizzazione:

- StandardScaler
- MinMaxScaler
- Normalizer
- MaxAbsScaler

10.5.1 Standard Scaler

Lo Standard Scaler è un Estimator. Questo Estimator può essere allenato utilizzando la funzione fit per produrre un modello, lo StandardScalerModel, un Trasformer che trasforma un vettore colonna di un DataFrame messo in input normalizzando ogni feature del vettore per avere un'unità di deviazione standard. Vediamo ora il codice:

```

1 from pyspark.mllib.linalg import Vectors
2 from pyspark.ml.feature import VectorAssembler
3 from pyspark.ml.feature import StandardScaler
4 # input and output folders
5 inputPath = "data/exampleDataAssembler.csv"
6 # Create a DataFrame from the input data
7 inputDF = spark.read.load(inputPath,\
8                             format="csv", header=True, inferSchema=True)
9 # Create a VectorAssembler that combines columns colB and colC
10 # The new vector column is called features
11 myVectorAssembler = VectorAssembler(inputCols = ['colB', 'colC'],\
12                                     outputCol = 'features')
13 # Apply myVectorAssembler on the input DataFrame
14 transformedDF = myVectorAssembler.transform(inputDF)
15
16 # Create a Standard Scaler to scale the content of features
17 myScaler = StandardScaler(inputCol="features", outputCol="scaledFeatures")
18

```

```

19 # Compute summary statistics by fitting the StandardScaler
20 # Before normalizing the content of the data we need to compute mean and
21 # standard deviation of the analyzed data
22 scalerModel = myScaler.fit(transformedDF)
23
24 # Apply myScaler on the input column features
25 scaledDF = scalerModel.transform(transformedDF)

```

Analizzando il codice, quello che viene fatto è inizializzare lo Standard Scaler con il metodo `StandardScaler(inputCol,outputCol)` in base alla colonna che vogliamo prendere in input e all'output che vogliamo ritornare, poi utilizziamo il metodo `fit` per allenare lo Standard Scaler con i dati presenti nel dataframe della colonna specificata per produrre uno `StandardScalerModel`. Essendo un Transformer possiamo poi usare il metodo `transform` per ottenere un nuovo Dataframe.



colA	colB	colC	features
1	4.5	True	[4.5,1.0]
2	0.6	True	[0.6,1.0]
3	1.5	False	[1.5,0.0]
4	12.1	True	[12.1,1.0]
5	0.0	True	[0.0,1.0]

colA	colB	colC	features	scaledFeatures
1	4.5	True	[4.5,1.0]	[0.903,2.236]
2	0.6	True	[0.6,1.0]	[0.120,2.236]
3	1.5	False	[1.5,0.0]	[0.301, 0.0]
4	12.1	True	[12.1,1.0]	[2.428,2.236]
5	0.0	True	[0.0,1.0]	[0.0 ,2.236]

Figura 12: StandardScaler

10.6 Gestione delle variabili categoriali

Frequentemente i dati di input sono caratterizzati da attributi categorici ovvero stringhe. Gli algoritmi di classificazione e regressione di SparkMLlib lavorano solo con valori numerici. Quindi le colonne categoriche vanno mappate su valori double. Le colonne categoriali devono essere convertite in valori numerici tramite **StringIndexer**. Per tornare ai valori originali dopo la predizione, si utilizza **IndexToString**. **String Indexer** è un Estimator che ritorna tramite il metodo `fit` un Transformer del tipo `StringIndexerModel` che codifica le label di una colonna in indici. Ogni valore della colonna è mappato un valore integer.

Il metodo `StringIndexer(inputCol,outputCol)` presenta come parametri `inputCol` che è il nome della colonna da mappare in un set di integer e `OutputCol`, la nuova colonna risultante. Il metodo `fit` su un Dataframe risultante del metodo `StringIndexer` ritorna un `StringIndexerModel`. Sullo `StringIndexerModel` possiamo invocare il metodo `transform` per creare un nuovo Dataframe che avrà la nuova colonna `outputCol`.



id	category
1	a
2	b
3	c
4	c
5	a

id	category	categoryIndex
1	a	0.0
2	b	2.0
3	c	1.0
4	c	1.0
5	a	0.0

Figura 13: StringIndexer

Esempio:

```

1 from pyspark.ml.feature import StringIndexer, IndexToString
2
3 data = [(0, "cat"), (1, "dog"), (2, "cat"), (3, "bird")]
4 df = spark.createDataFrame(data, ["id", "category"])
5
6 # Codifica delle categorie
7 indexer = StringIndexer(inputCol="category", outputCol="categoryIndex")
8 indexed = indexer.fit(df).transform(df)
9
10 # Conversione inversa
11 converter = IndexToString(inputCol="categoryIndex", outputCol="originalCategory",
12                           labels=indexer.fit(df).labels)
13 restored = converter.transform(indexed)
14 restored.show()

```

Il metodo `IndexToString` è il metodo simmetrico allo `StringIndexer`, è un `Transformer` che mappa una colonna di indici in una colonna di stringhe. I metodi di classification ritornano la versione integer per cui dobbiamo ri-mappare i valori in stringhe leggibile per l'umano. Il metodo `IndexToString(inputCol, outputCol, labels)` presenta come parametri `inputCol` ovvero la colonna originaria, `outputCol`, la nuova colonna, `labels` ovvero la lista originaria di stringhe su cui mappare i valori integer della colonna che vogliamo mappare. Il metodo `transform` su un `IndexToString` che ha come parametro un `DataFrame`, ritorna un nuovo `Dataframe` con la nuova colonna e le colonne originarie.



id	category
1	a
2	b
3	c
4	c
5	a

id	category	categoryIndex	originalCategory
1	a	0.0	a
2	b	2.0	b
3	c	1.0	c
4	c	1.0	c
5	a	0.0	a

Figura 14: `IndexToString`

10.7 SQLTransformer

L'`SQLTransformer` è un transformer che implementa un'istruzione SQL. L'`SQLTransformer` esegue una query sul `DataFrame` messo come parametro, ritornando un nuovo `DataFrame` derivante dalla query SQL.

```

1 from pyspark.sql.types import *
2 from pyspark.ml.feature import SQLTransformer
3 #Local Input data
4 inputList = [(1, "This is Spark"),\
5              (2, "Spark"),\
6              (3, "Another sample sentence of words"),\
7              (4, "Paolo Rossi"),\
8              (5, "Giovanni")]
9 # Create the initial DataFrame
10 dfInput = spark.createDataFrame(inputList, ["id", "text"])
11 # Define a UDF function that counts the number of words in an input string
12 spark.udf.register("countWords", lambda text: len(text.split(" ")), IntegerType())
13 # Define an SQLTransformer to create the columns we are interested in

```

```

14 sqlTrans = SQLTransformer(statement="""SELECT *,
15         countWords(text) AS numLines
16         FROM __THIS__""")
17 # Create the new DataFrame by invoking the transform method of the
18 # defined SQLTranformer
19 newDF = sqlTrans.transform(dfInput)

```



id	text
1	This is Spark
2	Spark
3	Another sample sentence of words
4	Paolo Rossi
5	Giovanni

id	text	numWords
1	This is Spark	3
2	Spark	1
3	Another sample sentence of words	5
4	Paolo Rossi	2
5	Giovanni	1

Figura 15: SQLTransformer

10.8 Algoritmi di classificazione

Apache Spark MLlib supporta diversi algoritmi di classificazione, tra cui:

- Regressione logistica (binomiale e multinomiale).
- Albero decisionale (*Decision tree*).
- Foresta casuale (*Random forest*).
- Alberi potenziati (*Gradient-boosted trees*).
- Percettrone multistrato (*Multilayer perceptron*).
- SVM lineare (*Linear Support Vector Machine*).

Tutti gli algoritmi seguono due fasi principali:

1. **Addestramento del modello** su un dataset di dati etichettati.
2. **Predizione delle etichette di classe** per nuovi dati non etichettati.

Gli input devono essere numerici, per cui le variabili categoriali devono essere convertite in valori numerici prima dell'applicazione degli algoritmi.

10.8.1 Input per i modelli di classificazione

Per addestrare un modello in Spark, l'input deve essere un **DataFrame** con almeno due colonne:

- **label**: rappresenta l'etichetta di classe da prevedere (valore numerico di tipo `double`).
- **features**: vettore di valori numerici rappresentante le caratteristiche del record (`pyspark.ml.linalg.Vector`).

10.8.2 Esempio: classificazione dei clienti

Supponiamo di voler classificare i clienti come "buoni" o "cattivi" sulla base di:

- **Reddito mensile.**
- **Numero di figli.**

Le categorie saranno mappate come segue:

- "Good customer" → 1.0.
- "Bad customer" → 0.0.

Utilizziamo quindi dei dati per fare il training:

CustomerType	MonthlyIncome	NumChildren
Good customer	1400.0	2
Bad customer	11105.5	0
Good customer	2150.0	2

Figura 16: Input iniziale

Ottenendo il seguente dataframe in output : Il modello viene addestrato utilizzando il dataset

label	features
1.0	[1400.0 , 2.0]
0.0	[11105.5, 0.0]
1.0	[2150.0 , 2.0]

Figura 17: Output

trasformato separato in etichetta e attributi che contribuiscono nella scelta dell'etichetta. Quindi prima di tutto dobbiamo creare un dataset con l'etichetta definita sulla base degli attributi. Basterà in questo caso utilizzare un Vector Assembler come Trasformator e creare attraverso transform il nuovo Dataframe.

```

1 from pyspark.ml.feature import VectorAssembler
2 from pyspark.sql import SparkSession
3
4 # Creazione della sessione Spark
5 spark = SparkSession.builder.appName("ClassificationExample").getOrCreate()
6
7 # Dati di esempio
8 data = [(1.0, 1400.0, 2.0),
9         (0.0, 11105.5, 0.0),
10        (1.0, 2150.0, 2.0)]
11 columns = ["label", "MonthlyIncome", "NumChildren"]
12
13 # Creazione del DataFrame
14 df = spark.createDataFrame(data, columns)
15
```

```

16 # Creazione della colonna features
17 assembler = VectorAssembler(inputCols=["MonthlyIncome", "NumChildren"], outputCol="
    features")
18 transformed_df = assembler.transform(df).select("label", "features")
19
20 transformed_df.show()

```

10.8.3 Logistic Regression

La **Logistic Regression** è un metodo molto utilizzato per predire risposte categoriche. Si basa sui modelli lineari generalizzati (Generalized Linear Models, GLM) e viene utilizzata per stimare la probabilità degli esiti. Il modello calcola le probabilità per ogni possibile classe dell'esito. Utilizza una funzione sigmoide (per esiti binari) o una funzione softmax (per esiti multiclass) per trasformare una combinazione lineare delle caratteristiche in una probabilità. Abbiamo visto prima come presi dei dati, possiamo creare un Dataframe in cui abbiamo due campi label (l'etichetta che verrà usata per identificare i vari record delle tabelle non identificate) e le features in cui abbiamo vettori di valori utilizzati poi per predire l'etichetta. Ad esempio dati i seguenti dati:

label	attr1	attr2	attr3
1.0	0.0	1.1	0.1
0.0	2.0	1.0	-1.0
0.0	2.0	1.3	1.0
1.0	0.0	1.2	-0.5

Otteniamo sulla base del codice prima espresso, un dataframe del genere con due attributi: Il file

label	features
1.0	[0.0,1.1,0.1]
0.0	[2.0,1.0,-1.0]
0.0	[2.0,1.3,1.0]
1.0	[0.0,1.2,-0.5]

Figura 18: Dataframe ottenuto dai dati etichettati

privo di etichette ha la stessa struttura del file dei dati di training. Vogliamo ora predire l'etichetta per ogni record dei dati non etichettati. Consideriamo quindi i seguenti dati come esempio:

label	attr1	attr2	attr3
	-1.0	1.5	1.3
	3.0	2.0	-0.1
	0.0	2.2	-1.5

Come vediamo abbiamo tutti i valori per ogni attributo tranne per label (etichetta) che sarà l'insieme dei valori da predire. Anche in questo caso andremo a creare un Dataframe per il rispettivo set di dati: Dopo l'applicazione del modello di classificazione, Spark ritorna un nuovo DataFrame contenente le stesse colonne del DataFrame di input, una nuova colonna chiamata predizione per ogni record non etichettato contenente la classe predetta e altre due colonne associate alla probabilità della previsione.

label	features
null	[-1.0,1.5,1.3]
null	[3.0,2.0,-0.1]
null	[0.0,2.2,-1.5]

Figura 19: Dataframe ottenuto dai dati non etichettati

label	features	prediction	rawPrediction	probability
null	[-1.0,1.5,1.3]	1.0	...	...
null	[3.0,2.0,-0.1]	0.0	...	...
null	[0.0,2.2,-1.5]	1.0	...	...

Figura 20: Dataframe ottenuto dalla previsione del modello predittivo

```

1 from pyspark.ml.classification import LogisticRegression
2
3 # Creazione dell'oggetto LogisticRegression
4 lr = LogisticRegression(maxIter=10, regParam=0.01)
5
6 # Addestramento del modello
7 model = lr.fit(transformed_df)
8
9 # Output del modello addestrato
10 print("Coefficients:", model.coefficients)
11 print("Intercept:", model.intercept)

```

10.8.4 Predizione su nuovi dati non etichettati

I nuovi dati devono essere strutturati come il dataset di addestramento, con una colonna **features** contenente i vettori delle caratteristiche.

attr1	attr2	attr3
-1.0	1.5	1.3
3.0	2.0	-0.1
0.0	2.2	-1.5

```

1 # Dati non etichettati
2 unlabeled_data = [(-1.0, 1.5, 1.3),
3                   (3.0, 2.0, -0.1),
4                   (0.0, 2.2, -1.5)]
5 unlabeled_columns = ["attr1", "attr2", "attr3"]
6
7 # Creazione del DataFrame
8 unlabeled_df = spark.createDataFrame(unlabeled_data, unlabeled_columns)
9
10 # Creazione della colonna features
11 unlabeled_df = assembler.transform(unlabeled_df)
12
13 # Predizioni

```

```
14 predictions = model.transform(unlabeled_df)
15
16 # Selezione delle colonne di interesse
17 predictions.select("features", "prediction").show()
```

Output:

```
+-----+-----+
|features          |prediction|
+-----+-----+
| [-1.0, 1.5, 1.3] |1.0      |
| [3.0, 2.0, -0.1] |0.0      |
| [0.0, 2.2, -1.5] |1.0      |
+-----+-----+
```

Le predizioni possono essere salvate in un file CSV:

```
1 predictions.select("features", "prediction").write.csv("output_predictions", header=
    True)
```